# No Code AI and Machine Learning: My Program Book

MIT Professional Education, delivered with Great Learning. Cohort January 2026 B. Second edition, with my own notes and work.

Prepared for Gilbert Paul Pelletier. Private study copy.
September 2026, second edition

## Contents


**Preface**

**Part 1. The program at a glance**

- 1.1 Identity card
- 1.2 MIT faculty behind the content
- 1.3 How the program was graded
- 1.4 The people you dealt with, and why it matters for revision
- 1.5 Your own archive

**Part 2. Your cohort timeline**

- 2.1 Enrollment and pre work (December 2025 to mid January 2026)
- 2.2 Core weeks (January 17 to May 4, 2026)
- 2.3 Completion

**Part 3. The curriculum, module by module**

- Module 0. Pre work: the world of data, and the mathematics behind AI
- Module 1. Introduction to the AI and Generative AI landscape
- Module 2. Data exploration: structured data, networks, and graphical models
- Module 3. Prediction methods: regression
- Module 4. Decision systems
- Module 5. Project week: machine learning classification. Hotel booking cancellations
- Module 6. Recommendation systems
- Module 7. Prediction methods: deep learning and neural networks
- Module 8. Computer vision methods

Module 9. Project week: prediction methods with deep learning and neural networks (Project 6)

Module 10. Generative AI foundations

Module 11. Business applications of generative AI

Module 12. Ethical and responsible AI

Self paced module A. Data exploration: unstructured data

Self paced module B. Data exploration: temporal data

**Part 4. Tools reference**

4.1 RapidMiner AI Studio 2025.1 (Altair)

4.2 KNIME Analytics Platform

4.3 Teachable Machine, Ikigai, and generative tools

4.4 The generative and agentic stack you actually used

**Part 5. Your projects and portfolio**

**Part 6. Formula sheet and glossary**

6.1 Formula sheet

6.2 Glossary

**Part 7. Self test and revision plan**

7.1 Twenty questions

7.2 Twenty more questions, taken from what the course actually examined

7.3 A thirty day plan to go one level deeper

**Appendix A. Sources used for this book**

**Appendix B. Reading list matched to the modules**

**Appendix C. Discrepancies between your own files**

# Preface

This book is a private companion to the program **No Code AI and Machine Learning: Building Data Science Solutions**, offered by MIT Professional Education and delivered online by Great Learning on the Olympus platform. You enrolled on December 29, 2025, started with the January 2026 B cohort (program code MIT NCAIML JAN'26 B), and received your digital certificate on May 26, 2026, followed by the CEU letter on June 3, 2026.

The book has three purposes.

1. Record what the program was: structure, timeline, people, tools, and grading, so you never have to search the portal or your inbox again.
2. Restate the content of every module in a form that suits your background (mathematics degree, options and futures trading, current study of AI and machine learning). Where the course spoke in pictures, this book adds the formula.
3. Give you a revision kit: a formula sheet, a glossary, self test questions, and a plan to go one level deeper on each topic.

**How it was built.** The Olympus course pages could not be opened from the environment where this book was written, so the first edition rested on three kinds of sources: the program emails in your inbox (offer letter, welcome, schedule notices, content release notices, deadlines, certificate), the public program pages and catalog descriptions published by MIT Professional Education and Great Learning, and standard references for the techniques themselves. Every date in Part 2 that comes from an email is marked as confirmed. Nothing in this book should be treated as an official transcript.

> **Second edition.** For this edition you shared your Google Drive archive of the program (the folder *MIT* with its eleven numbered course folders, the admission and certificate folder, and the loose files at the root). Every readable file in it was opened: the official brochure, offer letter, delivery schedule and orientation deck, the MIT lecture handouts and transcripts, the Great Learning mentor decks, case study decks and FAQs, your own notes, transcripts and summaries, and your three project submissions with their working files. The result is that the inferred dates of the first edition are now replaced by the official schedule, the module chapters now describe what the lectures actually said rather than the standard content of the technique, and each project chapter records what you actually built and the numbers you actually reported.
>
> **Reading the colors.** Text in green is new or corrected in this edition. Black text is unchanged from the first edition. Where a first edition statement was wrong, the correction appears in green next to it and the wrong statement was removed, so that everything you read in black is still believed correct. Appendix C lists the points where your own files disagree with each other, so that you can decide which number to trust.

> **What still could not be read.** Four items resisted every attempt: the 185 MB file *ai book 2.pdf* at the root of the archive (no text layer), the 143 MB *Lecture Slides.pdf* of the computer vision module, the two LinkedIn share cards of the yoga case study (images only), and the saved web page of the NovaMind solution workflow (mhtml format). None of your files contains a grade, a quiz score or grader feedback, so the marks you obtained per course remain unknown to this book.

---

# Part 1. The program at a glance

## 1.1 Identity card

| Item | Detail |
|---|---|
| Program | No Code AI and Machine Learning: Building Data Science Solutions |
| Institution | MIT Professional Education |
| Delivery partner | Great Learning (enrollment, payments, platform, support) |
| Platform | Olympus (olympus.mygreatlearning.com) |
| Cohort | January 2026 B, code MIT NCAIML JAN'26 B (program batch id 19881; your Olympus watermark code is 8H4K6FA53C; the cohort had over 200 learners from 14 or more countries and 48 industries, with an average of 17 years of work experience) |
| Program advisor | Amit Jha |
| Program managers | Juhi (welcome, December 29, 2025), then Ansuman (from January 2, 2026) (the orientation deck lists three program managers, Saumya Kesarwani, Nandini Vohra and Ansuman Hota, with Meher Bakshi, Senior Program Manager, and Shruti Gupta, Senior Manager, as the escalation line) |
| Program office email | office.ncai.mit@mygreatlearning.com |
| Mentor for Mentored Learning Sessions | Aditya Modak (Aditya of the Great Learning program team also hosted the orientation) |

| Item | Detail |
|---|---|
| Instructors for optional online classes | Anuj Saboo, Jatin Dawar (Jatin Dawar, Senior Machine Learning Engineer at TELUS, seven years of experience, ran the orientation content and the hands on tool sessions; Dr. Georg Huettenegger, Specialist Leader in Conversational AI, ran the January 18 no code tools session and recorded the RapidMiner case study videos; the January 25 introduction to data science session was given by a Great Learning Data Science Manager with nine years in industry) |
| Format | Recorded MIT faculty lectures, weekly Mentored Learning Sessions, hands on tool sessions, case studies, quizzes, projects, discussion forum |
| Duration | 12 weeks of core content plus pre work and two self paced modules (official program dates January 17 to May 3, 2026, per the CEU letter) |
| Stated workload | 11 modules, about 80 study hours, 6 to 12 hours per week (the brochure states that 8 CEUs represent more than 100 hours of learning effort) |
| Tools | RapidMiner AI Studio 2025.1 (Altair), KNIME, Teachable Machine, Ikigai, and generative AI tools (ChatGPT, Gemini, NotebookLM, DALL E) (in practice the generative modules used Google AI Studio with Gemini 2.5 Pro, NotebookLM, n8n with OpenAI models, Pinecone and Weaviate vector stores, DuckDuckGo search, Adobe Firefly, ElevenLabs and Qooly; you ran KNIME 5.9.0 on macOS for your projects) |
| Fee | USD 2,850 total, of which USD 1,000 registration paid December 29, 2025 (offer letter: registration USD 1,000 before December 30, 2025, first installment USD 1,850 before January 26, 2026, USD 100 discount for one time full payment; refund policy in section 1.3) |
| Credential | Digital certificate from MIT Professional Education via Accredible (credential.net), issued May 26, 2026 (completion date on the credential May 20, 2026; credential id fd9c86a2-db09-43f7-8a02-fe66161ec834; Accredible short id o5zwvgxy; issued as a W3C Verifiable Credential v2 and an Open Badge 3.0, both signed on July 5, 2026 with an Ed25519 signature) |
| Continuing education | CEU letter issued June 3, 2026 (public program pages state 8 CEUs) (the letter, signed by Bhaskar Pant, Executive Director of MIT Professional Education, awards 8 MIT CEUs for the program held January 17 to May 3, 2026) |

> **Admission paperwork.** The admissions offer letter dated December 29, 2025 was signed by Clara Piloto, Director of Global Programs and of Digital Plus Programs at MIT Professional Education. It made participation contingent on the registration fee, listed the fee schedule above, and stated the completion rule that runs through every other document: complete all courses within the cohort timelines with a minimum of 80 percent in each course. The brochure (internal stamp of September 24, 2025) adds the program's track record at the time: 96 percent completion rate, more than 4,300 alumni, more than 30 batches, 98 percent satisfaction, 83 percent of learners with more than eight years of experience, and an industry mix led by IT and technology (29 percent),

banking and insurance (12 percent) and healthcare (9 percent). Prerequisites were exposure to programming concepts and high school statistics and mathematics; no coding was required.

**Refund policy** (offer letter, Annexure 1): full refund if you dropped out within 7 days of enrollment and more than 42 days before launch; USD 250 cancellation fee if more than 42 days before launch but more than 7 days after acceptance; 50 percent of the fee 22 to 41 days before launch; 100 percent fewer than 22 days before launch; no refund for non engagement.

## 1.2 MIT faculty behind the content

The recorded lectures are built by MIT faculty from the Institute for Data, Systems, and Society (IDSS) and the Department of Electrical Engineering and Computer Science. Public program pages name the following faculty for this program.

- **Devavrat Shah**, professor of AI and decision systems at MIT, director of the Statistics and Data Science Center, and cofounder of Ikigai Labs. His research on recommendation systems and time series is the basis for the recommendation and temporal data modules.
- **Munther Dahleh**, professor and founding director of IDSS. Systems, networks, and decision making.
- **John N. Tsitsiklis**, professor of electrical engineering. Probability, optimization, and the mathematics of learning.
- **Caroline Uhler**, professor at IDSS and the Broad Institute. Graphical models and causal inference, which anchors the "networks and graphical models" part of data exploration.
- **Stefanie Jegelka**, professor of electrical engineering and computer science. Machine learning, deep learning, and optimization.

Mentored sessions were run by Great Learning industry mentors, not by the MIT faculty.

**Who taught which course.** The Olympus page "Meet Your Faculty" that you saved gives the exact mapping. The program consists of 12 courses; the pre work is delivered by Great Learning and the rest by MIT faculty through video lectures.

| Course | Faculty |
|---|---|
| Pre work: Introduction to Data Science and AI | Dr. Abhinanda Sarkar (Academic Director, Great Learning; PhD Statistics, Stanford; formerly MIT applied mathematics, IBM, GE), Dr. Bradford Tuckfield (Wharton PhD, author of data science books), Dr. Pavankumar Gurazada (IIM Lucknow PhD, author of *Marketing Analytics*), Viplove Raj Sharma (Director, Great Learning), Dr. Georg Huettenegger (Vienna University of Technology PhD, Conversational AI since 2016) |

| Course | Faculty |
|---|---|
| Introduction to AI Landscape | Prof. Devavrat Shah, Andrew (1956) and Erna Viterbi Professor, EECS; director of the Statistics and Data Science Center; cofounder of Celect (sold to Nike in 2019) and of Ikigai Labs, where he is Chief Technology Officer |
| Data Exploration, Structured Data | Prof. Caroline Uhler, Henry L. and Grace Doherty Associate Professor, EECS and IDSS; PhD Statistics, UC Berkeley; causal inference, representation learning, gene regulation |
| Prediction Methods, Regression | Prof. John N. Tsitsiklis, Clarence J. Lebel Professor, EECS and LIDS; director of LIDS 2017 to 2020; coauthor of *Introduction to Probability* and *Introduction to Linear Optimization*; member of the National Academy of Engineering |
| Decision Systems | Prof. Munther A. Dahleh, William A. Coolidge Professor, EECS; founding director of IDSS (July 2015 to June 2023); joined MIT in 1987; robust control, systemic risk in networked systems |
| Recommendation Systems | Prof. Devavrat Shah |
| Prediction Methods, Neural Networks, and Computer Vision Methods | Prof. Stefanie Jegelka, Associate Professor, EECS; CSAIL and IDSS; PhD from the Max Planck Institute in Tübingen and ETH Zurich; Sloan Fellowship, NSF CAREER, DARPA Young Faculty Award |
| Generative AI Foundations, Business Applications of Generative AI, Ethical and Responsible AI | Prof. Munther A. Dahleh |

The mentor decks themselves were prepared by Great Learning and reused across cohorts; a video link (youtube.com/watch?v=hPSd-vbUgcw) appears in the footer of several of them.

## 1.3 How the program was graded

From the program office notices:

- **Mentored Learning Sessions** are mandatory and carry an attendance score of 10 marks each. Attendance is recorded only if you join through the Olympus link. Optional online classes and project support sessions carry no attendance score.
- **Weekly course assessments** are short quizzes with a maximum of **two attempts**. The advice was to attempt only after finishing the videos and practice exercises.

- **Course projects** are submitted through Olympus with a deadline. Tool flows were provided for both RapidMiner and KNIME, and you could choose either tool.
- **Project Discussion Forum** contributions were scored (your scores were published on March 28, 2026).
- **Completed projects** were added automatically to your Great Learning ePortfolio.
- **Self paced modules** (Data Exploration for Unstructured Data and for Temporal Data) had optional quizzes that did not affect the grade.

> **The official rules** (orientation deck, orientation transcript, and the Learning Instruments pages of each module):
>
> - Certificate eligibility requires a minimum of 80 percent in each course individually.
> - Within a course, assessments carry 90 percent of the weight and attendance at the weekend Mentored Learning Session carries 10 percent. In a course with a quiz only, the quiz is the 90 percent. In the three courses with a project, the quiz counts 50 percent and the project 40 percent.
> - Course assessments are time based multiple choice quizzes, one per course, deadline driven. The Data Exploration assessment page you saved shows the format: 15 questions, 45 minute time limit, 90 marks, 2 attempts, available January 29 and due February 9, 2026, no pause and no resume once opened. The neural networks module lists its assessment as 10 questions in 30 minutes, so the length varied by course.
> - Projects are worth 40 marks each and there are three mandatory projects: Decision Systems (Project 1), Prediction Methods with Neural Networks (Project 2, called Project 6 in the office notice), and Business Applications of Generative AI (Project 3). Plagiarized submissions score zero and late submissions are not accepted.
> - Two project tracks existed: **Business Interpretation** (a slide deck with the tool screenshots already provided, where you replace the guiding questions by your answers) and **No Code Hands On** (you run and adapt the provided RapidMiner or KNIME flow and include your own screenshots). You chose the hands on KNIME track for Projects 1 and 2 and built the generative AI project on your own n8n server.
> - Test Your Understanding quizzes between videos, practice case studies, practice course assessments and FAQ sheets carried no weight.
> - All live sessions were recorded and posted on Olympus within about six hours.

## 1.4 The people you dealt with, and why it matters for revision

The program had two support layers. Great Learning handled the process (advisor, program manager, WhatsApp, Calendly calls) and the MIT content sat on top. When revising, the useful assets are the MIT lecture videos, the case study videos, the mentor session recordings, and the project problem statements. All of those live on Olympus under your login and remain accessible after completion; alumni retain platform access to the learning content.

> The orientation deck describes the support ecosystem as follows: support requests on Olympus through "Need Assistance" answered within 24 to 48 hours; an AI Content Mentor chat under every video (each course has its own context, answers may be inaccurate, and there is a "Reset Chat Context" option); the program office email for

candidature changes; the program manager as single point of contact by call or text during office hours; WhatsApp groups; peer discussion forums; and the Great Learning community of more than 10,000 members. A graduate of an earlier batch, Angelique, gave the practical advice that was repeated all cohort long: watch every video and do the exercises before the mentor session, attend the optional sessions, and use the program manager early.

## 1.5 Your own archive

The Drive folder you shared is organized by course, and this book follows that order. Knowing where each thing lives will save you time later.

| Folder | What it holds |
|---|---|
| Admission and certificates | Brochure (two copies), offer letter (two copies), the official 2026 delivery schedule, the orientation deck, the CEU letter, the Accredible certificate PDF and its two JSON credential exports, your three self made "credential triptych" print layouts, the Georg Huettenegger video summary on the future of no code, the faculty page, a Coursera certificate (Duke University, Oil and Gas Industry Operations and Markets, January 1, 2026, unrelated to MIT), and the unreadable *ai book 2.pdf* |
| 1 Pre work | Olympus pre work pages, RapidMiner and KNIME walkthroughs and fix notes, the January 18 session transcript (Georg), the orientation transcript, the January 25 Otter transcript (mixed English and French) and your three summaries of it, and your February 15 revision pack |
| 2 AI Landscape | Module outline, your compiled lecture document with the full transcripts of videos 1.1 to 1.4, the mentor deck with the Uber case, the Melbourne housing case (RapidMiner and KNIME), the KNIME Uber problem statement |
| 3 Data Exploration | MIT transcripts of the seven videos, Great Learning content pages, mentor deck (education PCA and country clustering cases), the two problem statements, the hypothesis testing FAQ, your video notes, your quiz answer key, the assessment instructions |
| 4 Regression | Your notes on all sixteen videos, the handouts 1.0 to 1.16, the FAQs on regression and on evaluation metrics, the mentor deck (ShowTime case), the two closed caption transcripts of February 14 and 15, the BigMart and Hospital LOS practice cases, your screenshots |

| Folder | What it holds |
| --- | --- |
| 5 Decision Systems | Your Project 1 folder (final deck, drafts, workflow notes, another learner's example, screenshots), the problem statement, FAQ, templates and track description, the handouts 1.1 to 1.8, the FAQ spreadsheets, the mentor deck (Sales Leads case), the Online Shopper and Credit Card Churn practice cases, your video notes |
| 6 Recommendation Systems | Your compiled notes, the mentor deck (Reader's Club case), the FAQ, the GoodBooks KNIME exercise, the Yelp and Movie RapidMiner cases, the optional singular value thresholding handout and video notes |
| 7 Neural Networks | Your Project 6 folder (HMEQ deck, two reports, two dashboards, assembled KNIME screenshots, your chat transcript with an AI assistant), the fifteen handouts, the lecture deck, four pre reads, FAQs, your video notes, the mentor deck (Fashion MNIST), the hands on session summary, the Insurance Charges practice case, the University Admission extra case |
| 8 Computer Vision | Your Yoga Pose Detection case study (two deck versions and charts), the CNN course notes, the CNN FAQ, the mentor deck and the Rice classification case, your Granola summary of the mentored session, your CNN study guide |
| 9 Generative AI | Your export of the module pages, Dahleh's lecture deck, your Netflix Challenge KNIME write up (English and French), the BigChaos Netflix Prize paper, the Browbake practice exercise with your media outputs and the web tool you built, the n8n web search agent mentor deck |
| 10 Business Applications | Your export of the module pages, the NovaMind mentor materials, the three practice cases, and your MediSure project folder (final PDF, seven deck versions, plan, test notes, Pinecone recipe, corpus PDFs) |
| 11 Ethics and tools | The mentor deck and the Fair Gen no code solution, the module reader, your recorded transcript of the seven Dahleh videos, the two lecture decks, and the RapidMiner and KNIME interface walkthroughs |

# Part 2. Your cohort timeline

Dates marked **(confirmed)** come from emails in your inbox. Dates marked **(schedule)** come from the official "2026 Delivery Schedule NCAI INTL" that you saved, and dates marked **(your file)** come from the modification dates and contents of your own files. The inferred dates of the first edition have been removed.

## 2.1 Enrollment and pre work (December 2025 to mid January 2026)

| Date | Event |
|---|---|
| Dec 29, 2025 **(confirmed)** | Advisor call with Amit Jha, offer for registration, payment of USD 1,000, welcome from program manager Juhi, Olympus access opened |
| Dec 30, 2025 **(confirmed)** | "Start your journey" email: Program Overview course and Pre work course released; orientation announced for Saturday January 17, 2026 |
| Jan 2, 2026 **(confirmed)** | Ansuman introduced as program manager. Note from the platform: case study videos use RapidMiner AI Studio version 2025.1, and learners were told to install that exact version |
| Jan 4, 2026, Sunday 11:00 EST **(confirmed)** | Optional online class "Intro to the World of AI" with Anuj Saboo (repeat of the December 13 session). Prerequisite: the pre work module "Mandatory Content: Introduction to the World of Data" |
| Jan 11, 2026, Sunday 12:00 EST **(confirmed)** | Optional online class "The Must Know Mathematics and Statistics Behind AI" with Jatin Dawar |
| Jan 17, 2026, Saturday **(confirmed)** | Orientation and online class "Intro to the World of AI" with Jatin Dawar. Recording posted the same day. Program formally starts |

> The official schedule lists five pre course webinars, all optional and non graded, on the self paced course "Introduction to Data Science and AI": Intro to the World of AI on December 13, January 3 and January 17, and The Must Know Mathematics and Statistics Behind AI on December 20 and January 10. The schedule dates the weekend, the emails the actual Sunday session, which is why January 3 and January 4 both appear. The pre work itself had three mandatory sections (Introduction to the World of Data, Introduction to RapidMiner, Introduction to KNIME) and four optional ones (applications in retail, healthcare and banking; the data science lifecycle; mathematics and statistics with a Naive Bayes demo, matrices and image representation; history of data science and AI). A short mandatory quiz on the three mandatory sections was due the weekend after orientation.

| Date | Event |
|---|---|
| Jan 18, 2026 **(schedule, your file)** | "Introduction to No Code Tools" live session by Dr. Georg Huettenegger. Your saved transcript records his advice: pick the cloud platform your organization already uses; H2O pioneered AutoML; specialized tools (Recombee, MonkeyLearn, Teachable Machine, Lobe) are simpler but narrow; Altair bought RapidMiner and renamed it AI Studio; KNIME is open source and can embed Python, R and Java; gigabytes and hundreds of variables are fine in these tools, terabytes and thousands of attributes push you to code; regulated industries need model risk management around the model. Poll results: the majority answered the three quiz questions correctly |
| Jan 25, 2026 **(schedule, your file)** | "No Code Tool support and QnA" weekend. Your Otter transcript of that Sunday (2 hours 3 minutes) is of the session "Introduction to Data Science" given by a Great Learning Data Science Manager: AI versus automation (a vending machine is automation), ML and deep learning as implementations of AI, model complexity must match data complexity, generative AI and hallucination, the three pillars (computer science, mathematics and statistics, domain knowledge), the data engineer, data analyst and data scientist roles, supervised versus unsupervised learning with a bank credit card example, clustering, dimensionality reduction and reinforcement learning. You wrote three summaries of it on January 26 and 27 (a prose "gros sommaire", a bullet version and a short résumé) |

## 2.2 Core weeks (January 17 to May 4, 2026)

The table below is the official delivery schedule. Content was released on a Thursday, the mentored session (MLS) took place on the following weekend, and the quiz was due on the Sunday after that, so two courses were always open at once. The first edition of this book placed the modules one week later than the actual release dates and invented a break in late February; both errors are corrected here.

| Course (code) | Content release | Assessment due | Session weekend | MLS | Optional session |
|---|---|---|---|---|---|
| AI Landscape (AIL) | Jan 22 | Feb 1 | Jan 31 | yes | Hands on RapidMiner |
| Data Exploration, Structured Data (DESD) | Jan 29 | Feb 8 | Feb 7 | yes | Hands on KNIME |
| Prediction Methods, Regression (PMR) | Feb 5 | Feb 15 | Feb 14 | yes | Hands on RapidMiner (Feb 15) |
| Decision Systems (DS) | Feb 12 | Feb 22 | Feb 21 | yes | Hands on KNIME |

| Course (code) | Content release | Assessment due | Session weekend | MLS | Optional session |
|---|---|---|---|---|---|
| Decision Systems, Project 1 | Feb 12 | Mar 8 | | | |
| Project week break | | | Feb 28 | | Project tool support (KNIME and RapidMiner) and hands on RapidMiner |
| Recommendation Systems (RS) | Feb 26 | Mar 8 | Mar 7 | yes | Hands on RapidMiner |
| Prediction Methods, Deep Learning and Neural Networks (PMDLNN) | Mar 12 | Mar 22 | Mar 21 | yes | Hands on KNIME (Mar 22) |
| Neural Networks, Project 2 | Mar 12 | Apr 19 | | | |
| Computer Vision Methods (CVM) | Mar 19 | Mar 29 | Mar 28 | yes | |
| Easter break week | | | Apr 4 | | |
| Project week break | | | Apr 11 | | Project tool support (KNIME and RapidMiner) and hands on RapidMiner |
| Generative AI Foundations | Apr 2 | Apr 19 | Apr 18 | | |
| Business Applications of Generative AI | Apr 9 | Apr 26 | Apr 25 | yes | Hands on KNIME |
| Business Applications, Project 3 | Apr 9 | May 10 | May 2 | | |
| Ethical and Responsible AI | Apr 16 | May 3 | May 2 | yes | Project support, KNIME |
| Self paced: Data Exploration, Unstructured Data | Apr 23 | | | | |

| Course (code) | Content release | Assessment due | Session weekend | MLS | Optional session |
|---|---|---|---|---|---|
| Self paced: Data Exploration, Temporal Data (DETD) | Apr 23 | | | | |

The schedule was marked tentative, and the emails show that the office moved two deadlines: the Project 2 deadline became April 20 (the office notice said 12:30 EST, the problem statement 2:30 PM) and the Project 3 deadline became May 4 (12:30 EST in the notice, 2:30 PM in the problem statement). The Business Applications assessment, due April 26 in the schedule, was due April 27 in the office notice. Mentored sessions were held on Saturday or Sunday in small groups, with times communicated after orientation; yours were on Saturdays at 13:30 EST when a notice mentions the time.

**Your own footprints in the core weeks** (from the dates and contents of your files):

| Date | What you did |
|---|---|
| Jan 30 | Compiled the AI Landscape lecture slides with the full transcripts of videos 1.1 to 1.4 and the curated resources into one study document |
| Feb 14 | Attended the regression MLS (Zoom closed captions saved, 14:49 to about 15:45), took two screenshots of the scikit learn cheat sheet and the bias variance slide |
| Feb 15 | Attended the hands on RapidMiner session on the ShowTime case (closed captions saved); you spoke ten times, once about a red port that fixed itself when you redid the connection; you also saved the mentor's Google Drive link and built a revision pack of the pre work material ("15 feb total review") |
| Feb 21 | Saved structured notes on the eight Decision Systems videos |
| Feb 27 | Saved structured notes on the Data Exploration videos and the thirteen customer segmentation case videos |
| Feb 28 | Screenshot during the project support session; a KNIME random forest pipeline explanation saved |
| Mar 3 to 8 | Built Project 1 in KNIME: draft deck on March 4, workflow steps note on March 7, final deck exported on March 8, one day before the deadline as you counted it |
| Mar 16 | Downloaded the recommendation mentor deck again |
| Mar 19 | Saved the notes on all neural network videos |

| Date | What you did |
|---|---|
| Mar 20 to 28 | Built Project 6 (HMEQ) in KNIME: chat with an AI assistant on March 21 to run and improve the teacher's workflow, dashboards and reports on March 27, final deck on March 28 at 05:14 |
| Mar 23 | Saved a timestamped summary of the hands on KNIME session on Fashion MNIST |
| Mar 28 | Printed a CNN study guide; the mentored computer vision session was summarized with Granola |
| Apr 4 | Wrote the Netflix Challenge KNIME workflow guide, first in French, then in English |
| Apr 12 | Saved the Project 6 chat transcript as a PDF |
| Apr 15 | Ran the Browbake generative marketing exercise (Gemini poster, Firefly video, ElevenLabs voice, Qooly merge) and built a single page web tool to reproduce the workflow; downloaded Dahleh's generative AI deck |
| Apr 19 | Exported the Generative AI Foundations module pages |
| Apr 25 | Exported the Business Applications module pages, including a transcript of the guardrails video recorded at 13:09 |
| Apr 27 to May 2 | Built the MediSure project: plan on April 27, Pinecone recipe and Workflow 2 tests on April 27 and 28, Workflow 3 tests on April 29, seven deck versions between April 29 and May 2, final PDF exported May 2 at 04:06 |
| May 4 | Recorded the seven Ethical and Responsible AI lectures in one 2 hour 8 minute transcript at 16:08 |

## 2.3 Completion

| Date | Event |
|---|---|
| May 20, 2026 **(credential)** | Completion date recorded on the Accredible credential (validFrom and completionDate fields) |
| May 26, 2026 **(confirmed)** | Digital certificate issued through Accredible. Your credential page is at credential.net under the ID that appears in the certificate email |
| Jun 1, 2026 **(your file)** | You produced the English and French LinkedIn share cards and comparison charts of your yoga pose case study |
| Jun 3, 2026 **(confirmed)** | CEU letter sent as a PDF by the program office |

| Date | Event |
|---|---|
| Jul 5, 2026 **(credential)** | Cryptographic proofs generated for the two verifiable credential exports (W3C VC v2 and Open Badges 3.0); you later assembled them into three print layouts titled "MIT Credential Triptych" |
| Jul 21, 2026 **(confirmed)** | Alumni survey from the MIT PE No Code AI and Machine Learning office |

# Part 3. The curriculum, module by module

Each module below follows the same structure: what the module taught, the mathematics under it (kept at the level you can use, not the level of a proof), how it looked inside the no code tools, the case study or project, and a short list of what to keep. In this edition each module also has three green sections: what the MIT lectures actually said (from the handouts and transcripts in your archive), what the mentor session and case studies did (with their numbers), and your own notes and work.

## Module 0. Pre work: the world of data, and the mathematics behind AI

**What it taught.** Two optional sessions and a self paced course set the vocabulary: data versus information versus decision, descriptive versus predictive versus prescriptive analytics, the difference between AI, machine learning, and deep learning, and how industries went from reporting to prediction. The mathematics session covered the pieces the later modules assume: descriptive statistics, probability, distributions, correlation, basic linear algebra (vectors, matrices, dot products), and the idea of a loss function.

**What you already had.** With a mathematics degree the content was elementary. The useful part was the mapping between course vocabulary and the mathematics you know:

- "Feature" means a coordinate of the input vector $x \in \mathbb{R}^d$; "label" or "target" means $y$.
- "Training" means choosing parameters $\theta$ to minimize an empirical risk $\frac{1}{n}\sum_i \ell(f_\theta(x_i), y_i)$.
- "Supervised" means $y$ is observed; "unsupervised" means it is not and the goal is structure in $x$ alone.
- "Overfitting" is the gap between empirical risk and expected risk, controlled through validation sets and regularization.

**Keep.** The train, validation, test split is the single most important discipline in the program. Every later module used it inside the tools.

> **What the pre work actually contained.** The Olympus pre work course had seven sections. The three mandatory ones were Introduction to the World of Data (origin of decisions, paradigms of data science and AI, defining data science, the need for mathematics and statistics, probability and descriptive statistics; four videos), Introduction to RapidMiner (installation, interface, importing data, preprocessing; seven videos) and Introduction to KNIME (installation, importing a workflow, interface; five videos). The optional sections covered applications (inventory optimization in retail, diabetes onset in healthcare, credit card fraud in banking), the data science lifecycle for e commerce, the mathematics behind AI (a Naive Bayes demo, matrices, a small recommendation demo, image representation, dimension reduction) and the history of the field from the 1940s to the 2020s.

**Tools as introduced.** RapidMiner began in 2001 as YALE (Yet Another Learning Environment) at the Technical University of Dortmund, is written in Java, and is built on a modular operator concept with nested operator chains. KNIME was presented as an open source, enterprise grade platform with a drag and drop interface, prebuilt models and easy production integration. The program offered the educational licence of RapidMiner AI Studio 2025.x (free for one year) and KNIME 5.4.x (free, no licence). Georg's practical rules: sign up for the educational licence through the link in the pre work PDF and never choose the licence server option; the KNIME videos show version 4.7 but newer versions work; the RapidMiner "Training Resources" folder appears empty until you download the training repository from the RapidMiner academy video. Your February 15 revision pack also kept the fix notes for two RapidMiner errors on the marketing dataset: "Cannot parse nominal to date" (set the date format and locale in the Nominal to Date operator) and the changed `date_diff` behavior in newer versions.

**Your notes.** Your three summaries of the January 25 session are the earliest personal documents in the archive and they already show the habit that runs through the whole archive: record the session, let a transcription tool produce the text, then rewrite it yourself at three levels of detail. The Otter transcript switched between English and French, so sentences such as "Le résultat de l'interaction entre l'algorithm and the training data is called a model" sit next to the instructor's English. The definitions you kept are worth restating because they are the ones the mentors reused all cohort long: an algorithm is theory, a model is the learned result deployed to production; regression is a problem type and linear regression one algorithm for it; when labels are missing you can look for a public labelled dataset, use a pretrained model, label by hand, or fall back to unsupervised learning; reinforcement learning is what social media recommendation loops run on, with likes and shares as positive reward and skips as negative reward.

## Module 1. Introduction to the AI and Generative AI landscape

**What it taught.** A map of the field as of 2025 and 2026: classical machine learning (regression, classification, clustering), deep learning, and the generative wave (large language models, diffusion models for images, and agents that call tools). The lecture placed each of the later modules on that map and introduced the no code philosophy: the workflow is a directed graph of operators (load, clean, transform, model, evaluate, deploy) and the practitioner's job is to choose and connect the operators, not to write them.

**The mathematics under it.** Not much beyond the definitions above. One idea worth keeping precisely: a generative model estimates a distribution $p(x)$ or $p(x \mid c)$ and samples from it, whereas a discriminative model estimates $p(y \mid x)$ or a decision function. Everything in modules 2 to 9 is discriminative or descriptive; modules 10 and 11 are generative.

**In the tools.** First contact with the RapidMiner AI Studio design canvas (repository, operators, ports, process panel, results view) and with the KNIME workflow editor (nodes, node repository, execute, output tables). Both tools were presented as equivalent; project flows were provided in both.

**Keep.** The operator graph mental model. When you read a Python notebook today, you can translate it back into that graph: each cell is an operator.

**What the MIT lectures actually said.** Devavrat Shah's deck is titled "No code Artificial Intelligence: For the people, By the people" and has four parts.

1. *History and context.* AI (term from the 1960s, Minsky's prediction that machines would do any human work within twenty years) aims at the "muscle" (actuation) and the "mind" (learning from data) of humans and was first built from rules and decision trees. Statistics (Graunt's demographics in the 1660s, Bayes in the 1700s, least squares, the method of moments and p values in the 1800s) explains the *why* through experiment design, hypothesis testing and causal attribution. Machine learning (Arthur Samuel, 1959, the checkers player) focuses on prediction and leans on computation. The three are compared on the ideal gas law $PV = nRT$: the statistician tests whether $PV$ is proportional to $T$ (interpretable, falsifiable), the machine learner predicts $T$ from $P$ and $V$ (accuracy), the AI engineer keeps $T$ at a target by adjusting $P$ and $V$ (control, automation).
2. *Organizations, people, data.* Organizations are run by people in front office (marketing, sales, support) and back office (supply chain, accounting, HR) operations. Before computers a store manager read demand by talking to customers and looking at shelves (Inditex is the example); information systems and then business intelligence replaced ledgers; e commerce removed the customers and the shelves and scattered the data over five or more systems, which is why spreadsheets rule. The **data operator** reconciles six or more systems of record into a source of truth every day, runs scenario analyses, operationalizes decisions and tracks results, and spends most of the day reconciling rather than deciding. The desiderata: decide for the best return on investment without engineers, without changing the daily process, with tools that speak to people.
3. *Data operations in e commerce.* The daily question is how much of each SKU to order. Inventory is a derived quantity (purchases minus sales) spread across systems. The demo shows a forecast per SKU, an ordering suggestion ("about three days of inventory left, high stockout risk this week"), a human override ("I only have USD 11,000 available today, wait until tomorrow"), and a scenario simulation: last season about USD 160,000 of inventory was held per day with USD 911,000 of missed opportunity; a different policy holds USD 10,000 less per day and cuts the missed opportunity to USD 588,000; an aggressive policy cuts it to USD 221,000. The interpretable policy reads "for this SKU, when two days of inventory are left, order 23 days' worth".
4. *Data operations in finance.* Which receivables to collect and which payments to make, daily. A USD 100 invoice paid in four installments (30, 40, 20 and 9) must be matched; the AI proposes matches and flags exceptions, humans approve, and approvals retrain the model. Once the ground truth exists you get days sales outstanding, days payable outstanding, cash flow forecasts with confidence intervals, and what if scenarios: 30 day cash flow rising from USD 79 million to USD 132 million by changing collection and payment policies.

The phrase to remember from the transcript: "AI provides scale, speed, and consistency; humans provide context, ethics, creativity, and strategic oversight." The "self driving organization" is presented as the next milestone, with the human kept in the loop.

**Mentored session and case studies.** The mentor deck (79 slides) opened with a five question quiz (primary goal of AI is to mimic human behavior intelligently; ML learns rules from data and answers rather than applying rules to data; statistics contributes through causal inference; the ideal gas law matching). The case was **Uber Data Analysis**: 29,101 hourly pickup records for the New York boroughs from January to June 2015, 13 columns (pickups, borough, holiday, wind, visibility, temperature, dew point, pressure, precipitation over 1, 6 and 24 hours,

snow depth). The nine step EDA wheel (understand, summarize, univariate, bivariate, correlation, hypotheses, patterns, insights, recommendations) produced these results: 359 missing temperatures imputed with the mean; the timestamp parsed into day, week, hour, weekday and month; pickups heavily right skewed with a maximum near 8,000 per hour; temperature bimodal near 35 and 60 degrees Fahrenheit and strongly correlated with dew point; only 3.9 percent of days were holidays; weather has almost no correlation with pickups; demand rose steadily so that June was about 2.8 times January; Manhattan dominates, followed at a distance by Brooklyn and Queens; the daily profile rises from 5 to 10 in the morning, dips before noon, and peaks at 19 to 20; Friday and Saturday demand stays high around midnight; Monday evenings are oddly low. Recommendations: staff the weekday commute peaks and Saturday nights, grow Brooklyn, Queens and the Bronx, and collect fleet and price data to build hourly demand and pricing models. The same problem statement was issued for KNIME for the office hours.

The **Melbourne Housing** practice case (Real Broker, 27,114 sales, 16 columns) taught the RapidMiner EDA workflow: import, mark "missing" and "inf" as missing in BuildingArea, statistics view, histograms, correlation matrix, scatter and bar charts. Findings: 77 percent of prices between 0.5 and 2 million AUD (average 1.05 million), 63 percent of houses 5 to 15 km from the central business district, 98 percent in metropolitan regions, 45 percent with three bedrooms, BuildingArea 61 percent missing; price rises with rooms, bedrooms, bathrooms and parking, falls with distance (sharply beyond 15 to 20 km) and with recent construction; distance and year built follow the city's radial development until about 1981. A KNIME version of the same case, with a walkthrough document, was also provided.

**Your notes.** Your compiled document of January 30 (the slide text, the platform summaries, the full transcripts of videos 1.1 to 1.4 under your heading "TEXTE from the teacher from the video", and the curated resources) is the most complete record of any lecture in the archive. Its curated resources are listed in Appendix B.

## Module 2. Data exploration: structured data, networks, and graphical models

**What it taught.** How to look at a table before modeling it. Summary statistics, missing values, outliers, distributions, correlation, and then three families of unsupervised methods:

1. **Clustering**: K means, K medoids, and Gaussian mixture models.
2. **Dimensionality reduction**: principal component analysis (PCA) and t SNE.
3. **Networks and graphical models**: representing relationships between variables or entities as a graph, and reading conditional independence from the graph structure.

Correction: the module as delivered was titled "Data Exploration: Structured Data" and did not contain a networks and graphical models section. Its actual topics were data collection and hypothesis testing, PCA and t SNE, clustering with K means, K medoids and Gaussian mixture models, and two additional content pages on hierarchical clustering and DBSCAN. The graphical model paragraph below is kept because it is the mathematics behind Caroline Uhler's research, but it was not examined.

**The mathematics under it.**

*K means* minimizes within cluster variance:

$$\min_{S_1,\dots,S_k} \sum_{j=1}^{k} \sum_{x \in S_j} \lVert x - \mu_j \rVert^2, \qquad \mu_j = \frac{1}{|S_j|} \sum_{x \in S_j} x.$$

Lloyd's algorithm alternates assignment and mean update and converges to a local minimum. K medoids replaces the mean by an actual data point, which makes it robust to outliers and lets you use any distance. A Gaussian mixture model assumes $p(x) = \sum_j \pi_j \, \mathcal{N}(x; \mu_j, \Sigma_j)$ and fits it by expectation maximization; the soft responsibilities $r_{ij} = \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j) / \sum_l \pi_l \mathcal{N}(x_i; \mu_l, \Sigma_l)$ are what K means hardens to 0 or 1.

*PCA* finds the orthonormal directions of maximal variance. With centered data matrix $X$ and covariance $C = \frac{1}{n} X^\top X$, the principal components are the eigenvectors of $C$ ordered by eigenvalue, equivalently the right singular vectors of $X$. The fraction of variance explained by the first $m$ components is $\sum_{i \le m} \lambda_i / \sum_i \lambda_i$. That ratio is the number the tools display as the "scree" or "cumulative variance" plot.

*t SNE* keeps neighborhoods rather than variance: it converts pairwise distances to probabilities $p_{ij}$ in high dimension and $q_{ij}$ in two dimensions (Student t kernel) and minimizes the Kullback Leibler divergence $\sum_{i \neq j} p_{ij} \log(p_{ij} / q_{ij})$ by gradient descent. Two consequences the course stressed: distances between t SNE clusters have no metric meaning, and the result depends on the perplexity setting and the random seed.

*Graphical models*: an undirected graph over variables where a missing edge between $i$ and $j$ encodes conditional independence $X_i \perp X_j \mid X_{\text{rest}}$. For Gaussian data this is read from the zeros of the precision matrix $\Sigma^{-1}$, which is the link to Caroline Uhler's research area.

> *Hypothesis testing, as the lecture did it.* Under the control model each of $n = 31{,}000$ women dies of breast cancer with probability $p = 0.002$, so the death count is $\mathrm{Bin}(31000, 0.002)$, centered at 62 (the lecture says 63, the observed control count) and approximately normal. The p value is $\Pr(X \le 39)$ under that model, which is $0.0012$; since it is below the 5 percent cutoff, the hypothesis that treatment and control are the same is rejected. The direction of the tail follows the quantity you care about: to the left for deaths and defects, to the right for sales and returns.
>
> *Silhouette score* (content page 1.9): for point $i$ with mean distance $a_i$ to its own cluster and mean distance $b_i$ to the nearest other cluster, $s_i = (b_i - a_i) / \max(a_i, b_i) \in [-1, 1]$; values near 1 mean well clustered, values near $-1$ mean probably the wrong cluster.
>
> *Gaussian mixtures and BIC* (video 1.7): the number of components can be chosen by the Bayesian information criterion, $\mathrm{BIC} = k \ln n - 2 \ln \hat{L}$, which penalizes the parameter count $k$; the lecture used it as the model based analogue of the elbow.

**In the tools.** RapidMiner operators: Retrieve, Replace Missing Values, Normalize, Detect Outlier, Correlation Matrix, k Means, PCA, and the visualization tab. KNIME nodes: Statistics, Missing Value, Normalizer, k Means, PCA, Scatter Plot.

**Case studies.** Customer segmentation for marketing, where clusters became personas, and a network view of who buys with whom.

**Keep.** Standardize before K means and PCA; both are scale dependent. Choose $k$ with the elbow of the within cluster variance and with silhouette scores, then sanity check clusters against business meaning.

> **What the MIT lectures actually said.** Caroline Uhler's seven videos (about 1.5 hours) and the two Great Learning content pages:
>
> - *1.2 Data collection, the HIP breast cancer study.* The question "does offering mammography save lives" is the same type of question as "does the new process reduce defects" or "does the campaign raise sales". Always have a control group and a treatment group and assign at random, otherwise a confounder (the Monday crew versus the Tuesday crew) explains the difference; randomization is A/B testing. The 1960s HIP study in New York randomized 62,000 women aged 40 to 60: 31,000 were invited to mammograms four times a year, 31,000 got standard insurance. Of the invited, 20,200 were screened and 10,800 refused. Breast cancer deaths per 1,000: screened 1.1 (23 deaths), refused 1.5 (16), whole treatment group 1.3 (39), control 2.0 (63). Deaths from other causes per 1,000 were 21 for the screened and 38 for the refusers, proof that the two subgroups are different people (education correlates with health), so the only valid comparison is the whole treatment group against the control group. The advertising analogue: people who click an ad differ from people who only see it. The study answers "does *offering* mammography help", which is also the policy one would implement.
> - *1.3 Hypothesis testing.* The binomial model and p value above.
> - *1.4 PCA.* High dimensional data (the genomic map example) projected on the directions of greatest variance; PCA is unit dependent (centimeters versus feet), so use the correlation matrix or standardize first.
> - *1.5 t SNE.* Objective is the KL divergence between neighborhood probabilities; initialize with PCA; the problem is non convex and suffers crowding; on 1,800 handwritten digits in 64 dimensions PCA gives one blob and t SNE gives ten clusters.
> - *1.6 and 1.7 Clustering, K means, PAM, GMM.* Within group sum of squares, the iterative process, choosing $k$ with the elbow, PAM (partitioning around medoids) as the robust version, Gaussian mixtures with the BIC.
> - *1.8 and 1.9 content pages.* Agglomerative clustering (single, complete and average linkage, cut the dendrogram at a big drop), DBSCAN (density based, related to single linkage, finds elongated clusters, minimum points parameter for noise), the silhouette plot, and the advice to run several methods because the cluster shapes are unknown at the outset; t SNE can also initialize K means.
>
> **Mentored session and case studies.** The mentor deck opened with a nine question quiz (dimensionality reduction; the three reasons for it, curse of dimensionality, feature selection and computational cost; the PCA objective; the five PCA steps, standardize, covariance, eigenvectors and eigenvalues, sort, select by cumulative variance; clustering maximizes within cluster homogeneity; Euclidean distance; clustering suits large unlabeled data; the elbow method; K means is sensitive to outliers). Two cases followed.
>
> - **Education data (PCA).** 777 US colleges, 18 columns, no missing values; PhD and Grad_Rate had one value each above 100 percent and were capped; Apps, Accept, Enroll and F_Undergrad were right skewed. After

standardizing, 4 principal components explained about 70 percent of the variance (the first two about 58 percent), a 76 percent reduction in dimensionality for a 30 percent loss in variance; weights above 0.25 in absolute value were read as significant. PC1 loads on Top10perc, Top25perc, Outstate, PhD, Terminal and Expend (premier colleges), PC2 on Apps, Accept, Enroll and undergraduate counts (large, easier colleges), PC3 on personal spending and books with a low student to faculty ratio, PC4 on low faculty education with a high graduation rate.

- **Socio economic country data (clustering).** 167 countries, 10 columns (child mortality, exports, health, imports, income, inflation, life expectancy, fertility, GDP per capita), no missing values; most variables right skewed except life expectancy. K means gave no clear elbow between 2 and 8 and produced clusters of 112, 51 and 5 countries (average within centroid distance 0.578), a sign of outlier sensitivity; K medoids showed drops at 2 and 4 and gave four clusters of 34, 39, 68 and 26 (distance 0.667). Profiles: cluster 0 highest child mortality and lowest income (Afghanistan, Uganda, Zambia); cluster 3 developing with the highest inflation (Indonesia, Philippines, Venezuela); cluster 1 trade heavy with low inflation (Singapore, Vietnam, Malaysia); cluster 2 developed (USA, UK).
- **Customer segmentation (RapidMiner, thirteen videos by Georg).** A marketing campaign table of more than 2,000 customers and 27 attributes: income near normal with 83 percent between 20,000 and 80,000, 63 percent in relationships, 1 percent complained in two years, recency capped at 99; features engineered (age, children, total spend, spend shares, purchases, promotions accepted, tenure); outliers in Year_Birth removed; K means with $k = 3$, then with PCA first (17 components explain everything, 7 explain over 80 percent, the first about 43 percent); elbows at 3 and 8, a five cluster model judged the most useful; profiles such as low income with many web visits and deals versus high income with catalog purchases and expensive wines.

**Your notes and work.** Your saved answer key for the module assessment (the file "MIT Project Quiz Answers New") lists your fifteen answers: a 5 percent significance level means a 5 percent chance of rejecting a true null; K medoids statements 1 and 2; scale before K means to give features equal importance; PCA extracts important features and reduces dimensionality; t SNE is preferable to PCA on a limited machine for 1,000 rows and 8 features; three clusters from the elbow plot shown; two components for 75 percent of variance; Euclidean distance; K means minimizes the within cluster sum of squares; the elbow method or the silhouette score chooses $k$; and, for the marketing benefit question, "all of the above" with your own note "mistake corrected, original answer was too narrow". No score is recorded in the file. Your video notes of February 27 cover every lecture and all thirteen case videos with timestamps.

## Module 3. Prediction methods: regression

**What it taught.** Predicting a continuous target: simple and multiple linear regression, the meaning of coefficients, goodness of fit, residual diagnostics, regularization (ridge and lasso), and the extension to logistic regression as a bridge to classification.

**The mathematics under it.**

Ordinary least squares solves $\min_\beta \|y - X\beta\|^2$ with closed form $\hat\beta = (X^\top X)^{-1} X^\top y$. Coefficients are partial effects, holding the other columns fixed, which is why multicollinearity inflates their variance (variance inflation factor). Ridge adds $\lambda\|\beta\|_2^2$ and yields $\hat\beta = (X^\top X + \lambda I)^{-1} X^\top y$, shrinking without zeroing; lasso adds $\lambda\|\beta\|_1$ and produces exact zeros, so it selects features.

Fit metrics: $R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$, adjusted $R^2$ penalizes the number of predictors, and the course preferred RMSE and MAE on a holdout set over any in sample statistic. Logistic regression models $\Pr(y = 1 \mid x) = \sigma(x^\top \beta)$ with $\sigma(z) = 1/(1 + e^{-z})$ and is fit by maximizing the log likelihood $\sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$; each coefficient is a log odds ratio.

> *Standard errors and intervals, as Tsitsiklis taught them.* Each estimate $\hat\theta_j$ is a random variable, approximately normal, unbiased, with spread equal to its standard error; the 95 percent confidence interval is $\hat\theta_j \pm 2\,\mathrm{SE}_j$; the interval, not the parameter, is the random object, and it captures the truth in 95 percent of repetitions. Reject $\theta_j = 0$ when the interval excludes zero (the Wald test); the p value is the probability of data at least as extreme under the null, not the probability that the null is true. Not rejecting can mean the null is true, the effect is small, or the sample is too small. Two sources of prediction error exist: the idiosyncratic noise, which no model removes, and the estimation error, which shrinks with $n$ and gives confidence bands of varying width along $x$. Heteroskedasticity breaks the standard error formulas (weighted least squares needs known variances), multicollinearity inflates them, and the bootstrap (resample with replacement, refit, look at the histogram of the coefficient) checks both the standard errors and the normality assumption.
>
> *Regularization* (optional video 1.16): with $p$ features the ridge estimate is $\hat\beta_\lambda = \arg\min \|y - X\beta\|^2 + \lambda\|\beta\|_2^2$; $\lambda$ is a hyperparameter chosen on validation data exactly like the number of features.

**A trading reading.** A linear regression of returns on factors is exactly this operator with factors as columns. The course's warnings about extrapolation, leverage points, and lookahead in the train test split map directly to overfitting a strategy on in sample data.

**In the tools.** RapidMiner: Set Role (label), Split Data, Linear Regression, Apply Model, Performance (Regression). KNIME: Partitioning, Linear Regression Learner, Regression Predictor, Numeric Scorer. Both display coefficients, p values, and the residual table.

**Case studies.** Price or demand prediction from tabular drivers, with a walk through of reading the coefficient table and the residual plot.

**Keep.** Always compare against a trivial baseline (predict the mean). A model that does not beat it on the holdout is noise.

> **What the MIT lectures actually said.** John Tsitsiklis's sixteen videos, with the handouts 1.0 to 1.16:
>
> - *1.0 to 1.3.* Regression as the cornerstone of supervised learning; predict a scalar $y$ from a vector $x$ with a function $g$ chosen to minimize mean squared error over $n$ records and $m$ attributes; a curve through every point has zero training error and is useless (overfitting), so restrict to linear predictors and solve by ordinary least squares, which is fast even in high dimension. The running example is the advertising dataset: 200

markets, budgets for TV, radio and newspaper, and sales. The multiple regression gives an unexpected negative newspaper coefficient while the simple regression of sales on newspaper alone shows a positive relationship, the first lesson on partial effects.

- *1.4 Two interpretations.* Either linear regression is the best linear approximation of an unknown relation in a representative sample (prediction), or the world is truly linear plus independent zero mean constant variance noise and the estimator recovers the true parameters (modeling). Same formula, different assumptions, different claims.
- *1.5 to 1.9.* $R^2$ as explained variance (about 90 percent for the three channel model, much lower for newspaper alone); standard errors, intervals, tests and p values as summarized above; the summary video insists on precise use of "accept" and "reject" and on the mathematical assumptions behind every result.
- *1.10 What can go wrong.* Latent variables, heteroskedasticity, multicollinearity, and the chocolate consumption versus Nobel prizes example (wealth explains both).
- *1.11 and 1.12 Feature engineering.* Add omitted variables (market size, competitors); keep categorical variables as separate indicators and never as one ordered number; a variable's significance depends on which others are present (newspaper loses it once TV and radio are in); transformations (logarithms, products) and interactions (TV times radio) keep the problem linear in the augmented vector; rescale so coefficients are comparable; a degree 20 polynomial through 10 points is the picture of overfitting; prefer regularization and data driven selection to classical stepwise procedures.
- *1.13 to 1.15 Validation.* Training, validation and test sets; validation error first falls then rises with the number of features; K fold cross validation (K between 5 and 10) averages the mean squared error over folds and wastes less data than a single split; leave one out is the case $K = n$; the bootstrap estimates the sampling distribution of a coefficient (the TV coefficient histogram matches the theoretical normal). The closing message: linear regression is the foundation of the non linear models and neural networks to come, and good prediction does not imply a correct model.

**Mentored session (February 14) and hands on session (February 15).** The mentor deck's quiz: statistical models aim at inference and ML models at prediction; the best fit line minimizes the squared vertical distances; RMSE is sensitive to outliers; multicollinearity is detected with correlation heatmaps or the variance inflation factor; linearity with pair plots; simple models have high bias and low variance, complex models the reverse; shuffle before K fold. From your closed captions of the session: the scikit learn cheat sheet as the map of the four families; a delivery time example (Uber Eats) where the business wants the fit line above the cloud so that most actual times are below the promise, "the business problem defines the best fit line"; statistical models can be reliable with 100 or 500 observations while ML models need much more; more training data makes training and validation curves converge.

The case was **ShowTime viewership**: 1,000 pieces of content, 8 columns (visitors, ad impressions, major sports event, genre, day of week, season, trailer views, first day views as target). Findings: first day views nearly normal with mean about 0.5 million; trailer views highly right skewed and the strongest correlate; ad impressions uncorrelated with views (a side insight that the company might cut ad spend); Wednesday and Saturday releases do better than Friday although Friday is the most common release day (a hypothesis in the session: Friday releases

are watched after midnight and counted on Saturday); Sci Fi slightly higher, Summer slightly higher. Models with dummy encoding and a 70:30 split: all variables RMSE 0.049, MAE 0.038, adjusted $R^2$ 76.8 percent; after removing 14 insignificant variables 76.7 percent; K fold 78 percent; bootstrap 77.2 percent; leave one out RMSE 0.040, the best. Coefficients in millions of views: one million more visitors gives about 123,000 more first day views, one million more trailer views about 2,000, a major sports event about 60,000 fewer, a Summer release about 19,000 more than Fall. Recommendations: avoid sports event days, raise platform visitors, collect data on duration, episodes and cast.

In the RapidMiner session you built the same model step by step: Retrieve the OTT data; Nominal to Numerical with dummy coding and comparison groups (Friday, Others, Winter, one dropped category per nominal variable because the dummies of one variable sum to one and would be perfectly collinear); Multiply; Correlation Matrix; a Subprocess named "linear reg" with Set Role, Split Data (seed 1992, 0.7 and 0.3), Linear Regression, Multiply Model, two Apply Model and two Performance operators. Results: RMSE 0.048 on training and 0.053 on test, squared correlation 0.800 and 0.74. The mentor read the coefficient table with the star code (two stars means p below 5 percent) and separated the size of an effect (the coefficient) from whether it is real (the p value). Variants with feature selection by T test, iterative T test and collinearity elimination gave similar results.

**Practice cases.** *Hospital length of stay* (HealthPlus, 50,000 admissions, 15 columns; stays of 3 to 51 days with median 9; 70 percent gynecology, 73 percent female, 80 percent insured; 80:20 split; RMSE 3.074 days, MAE 2.130, adjusted $R^2$ 0.853; 16 insignificant variables removed with no change; wards D, E and C have the most visitors and could rent space to shops). *BigMart sales* (1,559 products in 10 stores, 2013 data; the reduced model reads Item_Outlet_Sales $= 15.245\times$ Item_MRP $+3293.372\times$ SupermarketType3 $+1881.728\times$ Type1 $+1572.786\times$ Type2 $+70.058\times$ Regular $-64.570\times$ SmallOutlet $-62.261\times$ Tier3 $-1695.039$, so a Type3 store sells 2.09 times a Type2 store).

**Your notes and work.** Your summaries of all sixteen videos (with a separate "pop up video learning notes" file), the two closed caption transcripts, the mentor's Drive link, and the screenshots are all in the regression folder. The transcript of February 15 records your only spoken intervention in the archive: after connecting Retrieve to Nominal to Numerical the port stayed red and the attribute list was empty, and it resolved when you redid the connection ("Okay, I just redid it, and it works").

## Module 4. Decision systems

**What it taught.** Classification as decision making: decision trees, random forests, gradient boosted trees, the confusion matrix, precision, recall, F1, ROC curves and AUC, class imbalance, and how to choose a threshold when the two kinds of error have different costs.

Correction: the delivered module covered decision trees, entropy and information gain, the Gini index, overfitting and pruning, bagging and random forests, and classification and regression trees. Gradient boosting, ROC curves and AUC were not part of the MIT lectures; the mentor deck mentions boosting and stacking only as names in the ensemble taxonomy. The paragraphs on them below remain valid mathematics but were not examined.

**The mathematics under it.**

A decision tree splits on the feature and threshold that most reduce impurity. With class proportions $p_c$ at a node, Gini impurity is $1 - \sum_c p_c^2$ and entropy is $-\sum_c p_c \log_2 p_c$; information gain is the parent impurity minus the weighted child impurity. Trees are high variance; a random forest averages many trees grown on bootstrap samples with a random subset of features at each split, which reduces variance without much bias. Gradient boosting instead fits each new tree to the negative gradient of the loss of the current ensemble, so it reduces bias step by step and needs a learning rate and early stopping.

Metrics from the confusion matrix (TP, FP, FN, TN): precision $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, recall $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, F1 $= 2PR/(P+R)$. The ROC curve plots recall against false positive rate as the threshold moves; AUC is the probability that a random positive scores above a random negative. When classes are imbalanced, accuracy is misleading and the precision recall curve is the better picture.

**Decision theory in one line.** If a false negative costs $c_{FN}$ and a false positive costs $c_{FP}$, predict positive when $\Pr(y=1 \mid x) > c_{FP}/(c_{FP}+c_{FN})$. That threshold rule is the whole reason the module is called "decision systems" rather than "classification".

> *Why bagging works* (video 1.7): if $B$ classifiers err independently with probability $q < 1/2$, the majority vote errs only when more than half of them err, so the error falls with $B$ (the biased coin analogy). Random forests add feature sampling at each split (for example 5 of 15 features) to make the trees less correlated, at the price of weaker individual trees; the number of features per split and the number of trees are the hyperparameters. *Cohen's kappa*, which KNIME's Scorer reports next to accuracy and which you used in both projects, is $\kappa = (p_o - p_e)/(1-p_e)$, the agreement between prediction and truth beyond what chance would give; the course's rules of thumb call 0.2 to 0.4 fair, 0.4 to 0.6 moderate, and 0.6 to 0.8 substantial.

**In the tools.** RapidMiner: Decision Tree, Random Forest, Gradient Boosted Trees, Cross Validation, Performance (Classification), and the ROC plot. KNIME: Decision Tree Learner and Predictor, Random Forest Learner, Scorer, ROC Curve.

**Keep.** Report precision and recall at the threshold the business will actually use, not accuracy at 0.5.

> **What the MIT lectures actually said.** Munther Dahleh's eight videos:
>
> - *1.1 Classification as a concept.* Binary and m ary classification; predictors minimize error, classifiers minimize misclassification; linear versus non linear classifiers; the confusion matrix; decision trees as Boolean questions on thresholds.
> - *1.2 and 1.3 Decision trees.* Internal nodes test attributes, leaves carry labels; simple, interpretable, little data preparation, rank features (income first in the loan example), testable by cross validation; but non robust to small data changes, the optimal tree is NP complete so greedy algorithms are used, and they overfit with many features and little data. The restaurant waiting example (12 points, 10 features): a random tree misclassified 5 of 12, a data driven tree 2 of 12; leaves are labelled by majority rule; the empirical error approximates the true error.

- *1.4 and 1.5 Entropy and information gain.* Split the data 80/20 or 70/30, choose at each node the feature that most reduces entropy (a fair coin has maximum entropy, a constant coin zero); "estimated waiting time" gives a large gain in the restaurant case while "alternative" gives none; the Gini index replaces the logarithm by $1 - p$ and favors larger groups over singletons.
- *1.6 Overfitting and pruning.* The Titanic dataset (891 passengers, 38 percent survival, 80/20 split): sex is the top split, then age and family size; training error keeps falling with depth while cross validation error rises; pruning (grow, then merge branches that do not change misclassification) is presented as unnatural, and ensembles as the direct answer.
- *1.7 and 1.8 Bagging, random forests, CART.* Bootstrap aggregating (Tin Kam Ho, Breiman and Cutler, Amit and Geman), majority voting, variance reduction, the loss of interpretability, feature sampling per tree, regression trees, and the remark that bagging also applies to Gaussian classifiers and neural networks.

**Mentored session and case study.** The quiz: trees solve classification and regression; root, leaf, branch and decision node; the split maximizes homogeneity; entropy is the non additive, computationally heavier impurity; ensembles combine base models; random forests vote; their disadvantage is interpretability; pruning reduces overfitting. The case was **Sales Leads Conversion** for ExtraaLearn (4,612 leads, about 30 percent converted): working professionals are 67 percent of conversions and students 4 percent; website first contact converts 45 percent against 11 percent for the mobile app; high profile completion 45 percent against 7 percent for low; referrals are 2 percent of leads but convert 68 percent; converted leads spend a median of about 800 seconds on the site against 300. Recall was declared the crucial metric because a missed customer costs more than a wasted call. Results (train and test accuracy, train and test recall): baseline tree 99.97, 81.49, 100, 69.01; pruned tree (Gini, depth 14, minimal leaf 10) 88.05, 86.77, 82.26, 78.93; baseline forest 99.97, 87.20, 100, 73.85; pruned forest 89.78, 86.77, 72.3, 64.65. The pruned tree won on test recall with a train test gap under 10 percent, the deck's rule of thumb for overfitting. Feature importance: time on website, occupation, last activity, page views and age carry over 80 percent of the impurity reduction.

**Practice cases.** *Online Shopper Intention* (12,330 sessions, 15.47 percent with revenue; PageValues is the root split; RapidMiner tree with information gain and depth 8: test accuracy 90.81 percent, precision 84.47, recall 77.85; bagging with 5 bootstraps trades precision for recall; random forest with 50 trees 91.89, 86.40, 80.78; in the KNIME walkthrough the tree reached 97.4 percent on training and about 88 percent on test with an F measure of 59.7 percent for the true class, and the forest 89.9 percent with F 63.1 percent, the lesson being that accuracy is unreliable under imbalance and F1 is the metric to watch). *Credit card churn* (Thera Bank: Platinum holders attrite at 35 percent, customers who contacted the bank most attrite most, 2 to 4 months of inactivity is the danger zone; drivers Total_Trans_Ct, Total_Trans_Amt, Total_Relationship_Count).

**Your notes.** Your February 21 notes summarize all eight videos with timestamps. The project folder holds another learner's Business Interpretation submission (26 slides, dated February 17) that you kept as a structural model; its template numbers (tree 99.65 percent train and 83.20 test; pruned forest 87.32 and 85.04) are the RapidMiner reference numbers, not yours.

## Module 5. Project week: machine learning classification. Hotel booking cancellations

**The problem.** A hotel chain wants to predict which reservations will be cancelled so that it can manage overbooking, deposits, and pricing. The dataset is the widely used hotel bookings table (lead time, arrival month, number of nights, market segment, deposit type, customer type, average daily rate, special requests, previous cancellations, and the binary label "is cancelled").

Correction: the actual dataset was the INN Hotels Group (Portugal) table with Booking_ID, numbers of adults, children, weekend and week nights, required car parking space, room type, lead time, arrival year, month and date, market segment type, repeated guest, previous cancellations and previous non cancelled bookings, average price per room, number of special requests, and booking_status as the target; there is no deposit type column in it.

**What you did.** Loaded and explored the data, treated missing values and outliers, encoded categorical fields, split the data, trained at least a decision tree and a random forest (and typically logistic regression as a baseline), compared them on holdout precision, recall, and F1, chose a model, and wrote business recommendations. The project was added to your ePortfolio on March 17, 2026 under the title "Predicting the cancellation of Hotel Bookings by customers".

> **What you actually submitted.** The final deck "Gilbert P. Pelletier Knime Decision System Assessment" (exported March 8, 2026, dated March 3 on the title slide, Hands on Tool Assessment track with KNIME, prepared for INN Hotels Group) is built on the official template. Your own notes say the workflow was: CSV Reader, Data Explorer for statistics and univariate distributions, Bar Chart, Box Plot and Scatter Plot for multivariate views, a 70 percent training and 30 percent test partition, then Decision Tree and Random Forest learners with pruning. No logistic regression was built.
>
> *Executive summary as you wrote it.* Lead time risk: bookings made 90 or more days in advance cancel much more often. Silent guest profile: guests with no special requests and no parking need are more likely to cancel. Online travel agency bookings cancel more than direct bookings. Higher average daily rates go with more cancellations. Recommendations: non refundable early bird discounts for far ahead bookings; automated pre arrival emails inviting guests to personalize their stay; a credit card at booking with an automatic charge 48 to 72 hours before arrival for bookings the model flags as high risk.
>
> *Business problem.* 32.8 percent of bookings are cancelled, which is the primary driver of revenue loss and creates uncertainty in inventory, pricing and staffing.
>
> *EDA.* Lead time is the strongest single predictor; guests with more special requests cancel less; a correlation matrix (KNIME Linear Correlation node) was used to select predictors.
>
> *Model performance, your numbers.*

| Model | Train accuracy | Test accuracy | Train recall (cancelled) | Test recall (cancelled) | Gap in accuracy | Gap in recall |
|---|---|---|---|---|---|---|
| Decision tree, default | 95.7% | 83.0% | 95.3% | 73.4% | 12.7 points | 21.9 points |
| Decision tree, pruned | screenshot only | screenshot only | | | "reduced overfitting" | |
| Random forest, default | 98.8% | 88.0% | 98.9% | 83.8% | 10.8 points | 15.1 points |
| Random forest, pruned | screenshot only | screenshot only | | about 84% | "almost the same on unseen data" | |

The default tree's test confusion matrix showed 243 missed cancellations, which you called the main risk. You used a 10 point train test gap as the overfitting threshold, applied pruning to both models, and selected the pruned random forest because recall for the cancelled class was the key metric: an empty room costs more than a false alarm that only triggers a manual review. Feature importance from the forest confirmed lead time and special requests. The pruned numbers exist only inside screenshots, so the "about 84 percent of cancellations detected" claim cannot be cross checked from text; it equals the unpruned forest's test recall of 83.8 percent (see Appendix C).

*Draft to final.* The March 4 draft had a more speculative executive summary (a 90 day "risk threshold" with a 60 percent higher cancellation probability, a 25 percent deposit only in peak periods, an engagement email 30 days before arrival) and still carried the template placeholders in the modelling section, so the models were built and written up between March 4 and March 8.

*What the problem statement asked for* (40 marks): problem definition and EDA (8), model performance without tuning (10), model performance with tuning (10), best model and metric choice (6), actionable insights (6). No grade or feedback exists in your archive.

**The lessons that generalize.**

- Lead time and deposit type dominate; long lead times and no deposit are the classic cancellation signature. (In your data the signature was long lead time, no special requests, online channel and high room price.)
- Random forests usually win on this dataset, but the tree's rules are what the hotel manager can act on.
- The cost of a missed cancellation (an empty room) is not equal to the cost of a false alarm (a guest wrongly treated as a risk), so the threshold must be set from those costs, exactly as in Module 4.

## Module 6. Recommendation systems

**What it taught.** How platforms recommend products, films, or content: popularity baselines, content based filtering, collaborative filtering (user based and item based), matrix factorization, and how to evaluate recommendations (precision at k, recall at k, RMSE on held out ratings). Your mentored session on this topic was on Saturday March 7, 2026.

**The mathematics under it.**

Let $R \in \mathbb{R}^{m \times n}$ be the user by item rating matrix, mostly missing. Item based collaborative filtering predicts $\hat{r}_{ui} = \sum_{j \in N(i)} s_{ij}\, r_{uj} / \sum_{j \in N(i)} |s_{ij}|$ using cosine or Pearson similarity $s_{ij}$ between item columns. Matrix factorization assumes $R \approx U V^{\top}$ with $U \in \mathbb{R}^{m \times k}$, $V \in \mathbb{R}^{n \times k}$ and solves

$$\min_{U,V} \sum_{(u,i)\ \text{observed}} (r_{ui} - u_u^{\top} v_i)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

by alternating least squares or stochastic gradient descent. The low rank assumption is the same one that underlies the singular value decomposition; the difference is that only observed entries enter the loss. Devavrat Shah's own work on collaborative filtering as matrix completion sits directly behind this lecture.

> *The lecture's own estimators.* Shah builds the module as a sequence of estimators of the likelihood matrix $L_{ij}$ (user $i$ likes item $j$), each one relaxing an assumption of the previous one.
>
> - *Averaging.* If all users are identical, each observed entry in column $j$ is a coin toss with bias $L_{ij}$ and the column average estimates it; the central limit theorem gives an error scale of $1/\sqrt{n_j}$, so a film rated 4 by 5 users is less reliable than one rated 3 by 500. The handout's combined row and column estimator is written $2L_{ij} = L_i + 1/\sqrt{n_i} + L_j + 1/\sqrt{n_j}$.
> - *Content based.* Learn $L_{ij} = f(x_i, y_j)$ from user features $x_i$ and item features $y_j$ by supervised learning; text (tips, reviews) becomes a word frequency matrix reduced by PCA; useful for new users or items.
> - *Clustering.* Compute an $N \times N$ similarity matrix (cosine, or mean centered cosine), reduce by SVD or PCA, run K means, and average inside the cluster. Worked example: users with ratings $(3, 4, *, 2)$ and $(*, 4, 5, 1)$ share items 2 and 4, so cosine $= (4 \cdot 4 + 2 \cdot 1)/\sqrt{(16+4)(16+1)} = 0.97$; after mean centering (means 3 and 2.5) the cosine is exactly 1.
> - *Collaborative filtering* as personalized clustering: for user $i$ and item $j$ keep the $k$ nearest users who rated $j$ (user user) or the $k$ items most similar to $j$ that $i$ rated (item item), and average; the combined estimator is $L_{ij} = \sum_{i' \in I_i,\, j' \in I_j} L_{i'j'} / (|I_i|\,|I_j|)$. When two users share no items, bridge through a third ("thy friend is my friend", iterative CF). It scales with approximate nearest neighbors, is incremental, and is interpretable ("GoodFellas because you liked The Godfather").
> - *Singular value thresholding* (optional 1.8). Fill missing entries with 0 (or with row and column averages), compute $A = USV^{\top}$, keep the top $r$ singular values and rescale by $1/p$ where $p$ is the observed fraction: $\hat{L}_{ij} = \frac{1}{p} \sum_{k=1}^{r} s_k u_{ik} v_{jk}$. Choose $r$ at the knee of the singular value plot; a smooth decay means the low

> rank assumption fails. With $r = 1$ and one known row and column, $L_{ij} = L_{i1}L_{1j}/L_{11}$, so $m + n - 1$ entries recover the whole $m \times n$ matrix; in general a rank $r$ matrix needs about $(m + n)r$ entries. The optimization view minimizes the squared error on observed entries only and is solved by alternating least squares: fix $V$, regress to get $U$; fix $U$, regress to get $V$; repeat.

**In the tools.** RapidMiner has a recommender extension with Item k NN and matrix factorization operators; KNIME builds the same pipeline with similarity and matrix nodes. The course also showed how a simple "customers who bought X also bought Y" rule can be produced from association rules (support, confidence, lift). (Correction: association rules were not part of the delivered module. The RapidMiner Recommender Extension version 5.1.2 and the Operator Toolbox had to be installed from the Marketplace before opening the case files; KNIME's exercise used the Spark Collaborative Filtering Learner.)

**Case studies.** Streaming and retail recommendation, with a discussion of cold start (new users or items with no history) and of the popularity bias that pure collaborative filtering produces.

**Keep.** Evaluate on held out interactions per user, not on a random split of ratings; otherwise you leak the user's taste into training.

> **What the MIT lectures actually said.** Shah's nine videos: social data processing (data generated actively through reviews and passively through transactions; social choice as the key question); recommenders as decision systems in two steps, predict then optimize under constraints; the Yelp dataset (about 200,000 businesses, 2 million users, 8 million known reviews out of 0.4 trillion possible pairs, a known fraction of $2 \times 10^{-5}$) and MovieLens; the prediction problem as matrix, tensor and time varying tensor completion; the warning that data is not collected at random and can be manipulated (people rate when they feel strongly, providers alter addresses) and that recommenders create information bubbles and feedback loops; the estimators above; other matrix completion problems (friend suggestion, game outcomes and Elo ratings, crowdsourcing, synthetic controls for California's Proposition 99 tobacco policy); and a hands on session on Ikigai with MovieLens where row and column averaging gave MAE 0.78 and MSE 0.94, a content based model MSE 1.34, clustering 1.26, collaborative filtering 0.86 (the best), and naive singular value thresholding with zeros a poor result that motivates the squared error formulation.
>
> **Mentored session and case studies.** The quiz covered the goal of recommenders, popularity based systems for new users, cluster based and content based systems, user based CF, and SVD as the approximation of the interaction matrix by $U\Sigma V^\top$. The case was **Reader's Club** (RapidMiner): average rating 7.629 on a 10 point scale; 8 the most common rating; only 16 percent of ratings at 5 or below; 87,652 unique books; Stephen King and Nora Roberts the most frequent authors; only 0.005 percent of user item pairs rated. Filters kept users with at least 50 ratings and books with at least 10. Results (RMSE, MAE, NMAE): item item 1.514, 1.120, 0.124; user user 1.562, 1.156, 0.128; SVD with 10 latent factors 1.896, 1.541, 0.171. Item item was recommended; the deployment diagram sends new users to the popularity model, active users to user user CF, and users seeking diversity to item item CF. The **Yelp** RapidMiner case (229,907 reviews, 8,315 businesses, 45,413 users, density 0.06 percent, 67 percent of ratings at 4 or above): tuned item item CF with 100 neighbors, cosine similarity and a 70:30 split gave RMSE 0.971 and MAE 0.755, beating user user (1.008) and SVD (1.083); a content based model on review text

used tokenization, stop word filtering, stemming, TF IDF and Cross Distances. The **Movie** case (24,319 ratings, 606 users, 1,000 movies, 80:20 split): item similarity best with RMSE 0.780 because there are many more items than users. The **GoodBooks** KNIME exercise (10,000 books, 981,756 ratings, Spark Collaborative Filtering Learner, 70:30 split with seed 1): MAE 1.032, RMSE 1.442 on a 1 to 5 scale, judged not a good model; the appendix showed that items present only in the test set produce NaN predictions and break the Spark scorer unless removed.

**Your notes and work.** Your compiled document "Notes Recommendation Systems" (803 lines) assembles the outline, video summaries, handouts, the Reader's Club problem statement, the FAQ and the six Yelp case video summaries. Your only additions are section labels ("handout:", "video:", "Mentored session: see document in document of the slides").

**Your own extension: the Netflix Challenge workflow.** On April 4, 2026, after reading the 2009 BigChaos paper on the Netflix Grand Prize (Töscher, Jahrer and Bell), you wrote a 14 page guide, first in French ("NETFLIX DEFI") then in English ("NETFLIX CHALLENGE"), for a 40 node KNIME 5.9.0 workflow on the 8,807 title `netflix_titles.csv` catalog. Because the catalog has no ratings, you built a content based recommender and mapped each of its seven branches to a section of the paper.

1. *Ingestion and cleaning*: CSV Reader, Missing Value ("Unknown" for director, cast, country; median for release year), String Replacer and String To Number for the duration column (or one String Manipulation with `toInt(regexReplace(`$duration$`, "[^0-9]", ""))`), Column Filter, Row Filter.
2. *EDA*: Statistics, Value Counter (movie versus TV show), GroupBy by year and by rating, Bar and Pie charts, Cell Splitter on country; the United States has 3,690 titles, India 1,046, the United Kingdom 806; most titles were added between 2016 and 2021 with a peak in 2019 to 2020, echoing the paper's observation that ratings cluster at the end of the period.
3. *Feature engineering*: one hot genres from `listed_in`, title age $= 2021 -$ release year, a Rule Engine for Short, Medium and Long duration classes, Category To Number for type.
4. *Content similarity*: concatenate genres, director, country and description; a Python Script node with scikit learn's `TfidfVectorizer` (English stop words, 5,000 features) and `cosine_similarity`; sort and keep the top N per title. You related this to the paper's item item Pearson and Spearman correlations and its movie KNN.
5. *Clustering*: Normalizer, PCA to 10 to 20 components, K means with $k = 8$ and a static seed, scatter plot, GroupBy per cluster (expected clusters such as US action, international dramas, Japanese anime, documentaries, children's content), mapped to the paper's clustering based bins.
6. *Recommendation*: Table Row To Variable for a reference title (Stranger Things), filter and sort the similarity row, exclude the title itself, keep 10, join metadata; a blending score $0.5\, s_{\text{tfidf}} + 0.3\, s_{\text{genre}} + 0.1\, s_{\text{year}} + 0.1\, s_{\text{country}}$ as a simplified version of the paper's final linear blend.
7. *Evaluation*: genre coverage, intra list diversity and visual inspection, since RMSE cannot be computed without ratings.

> Extensions you listed: SVD or ALS nodes from the KNIME Recommender extension if ratings become available, BERT embeddings instead of TF IDF, a learner based blend like the paper's bagged gradient boosted trees, deployment as a KNIME component, and K fold cross validation with X Partitioner and X Aggregator (the paper used 34 folds). The paper itself you read in full: 100,480,507 ratings on 17,770 films by 480,189 users, target RMSE 0.8563, and the finding that 18 well chosen results suffice to breach the 10 percent barrier.

## Module 7. Prediction methods: deep learning and neural networks

**What it taught.** From logistic regression to a multilayer perceptron: neurons, activation functions, layers, forward propagation, loss, backpropagation, gradient descent and its variants, epochs and batches, overfitting and dropout, and when a neural network beats a tree ensemble on tabular data (rarely) versus on images, text, and audio (usually). The hands on session on KNIME for this module was on Sunday March 22, 2026 with Jatin Dawar.

**The mathematics under it.**

A network with layers $h^{(l)} = \phi(W^{(l)} h^{(l-1)} + b^{(l)})$ and output $\hat{y} = h^{(L)}$ is trained by minimizing a loss such as cross entropy $-\sum_c y_c \log \hat{y}_c$ (with softmax output $\hat{y}_c = e^{z_c} / \sum_k e^{z_k}$) or squared error. Backpropagation is the chain rule applied layer by layer: $\delta^{(L)} = \nabla_z \ell$, $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(z^{(l)})$, and $\partial \ell / \partial W^{(l)} = \delta^{(l)} h^{(l-1)\top}$. Stochastic gradient descent updates $\theta \leftarrow \theta - \eta \, \hat{\nabla} \ell$ on mini batches; Adam rescales each coordinate by running estimates of the first and second moments of the gradient. ReLU $\phi(z) = \max(0, z)$ avoids the vanishing gradients of sigmoid activations in deep stacks.

> *The lecture's derivations, as Jegelka wrote them.* One hidden layer with ReLU: $\partial L / \partial w_{11} = x_1 \cdot 1[z_1 > 0] \cdot v_1 \cdot (F(x_i) - y_i)$, that is input, activation derivative, downstream weight, and error. A gradient is always a product of an error, some subsequent weights and some activation derivatives, which is why tiny weights give vanishing gradients and large weights exploding ones, and why initialization matters: He initialization draws weights from a zero mean Gaussian with variance $2/(\text{number of units})$ for ReLU. Weight decay: $w_{t+1} = w_t(1 - \eta\alpha) - \eta \, \partial L / \partial w$. Dropout: switch off each hidden unit with probability $1/2$ during training and scale the weights by $1/2$ at test time (in general keep rate $1 - r_d$ and scale by $1/(1 - r_d)$). Batch normalization standardizes the inputs of each layer to mean 0 and standard deviation 1. Adam combines momentum (a moving average of gradients) with AdaGrad's per weight step size from the squared gradients, and has a slight edge thanks to bias correction. The flu example of the building blocks handout: inputs stomach ache, fever, cough with weights $-1, 3, 3$ and threshold 5; $(1, 0, 1)$ gives $2 < 5$ (no flu) and $(0, 1, 1)$ gives $6 > 5$ (flu).

**In the tools.** RapidMiner: Neural Net operator (hidden layer sizes, learning rate, epochs) and the Deep Learning operator from the H2O extension. KNIME: the Keras integration nodes (Keras Network Learner and Executor) shown in the hands on session, plus the simpler RProp MLP Learner.

Correction: the hands on session and both projects used KNIME's RProp MLP Learner and MultiLayerPerceptron Predictor, not the Keras nodes. The provided project files were three RapidMiner processes (Basic Neural Network, Neural Network With Parameter Tuning, Neural Network Using GridSearch) and one KNIME workflow

(`KNIME_ANN_project.knwf`) that chains a baseline model, a Parameter Optimization Loop and the optimized model.

**Keep.** Scale inputs, start small, watch training and validation loss together, and stop when validation loss turns up. On tabular business data, a tuned gradient boosted tree is the benchmark a network must beat.

> **What the MIT lectures actually said.** Stefanie Jegelka's deck "Introduction to Deep Learning" and the fifteen handouts:
>
> - *Why networks.* Predictions from everything; a linear classifier compares $\theta^\top x$ to a threshold, a feature based one compares $\theta^\top \phi(x)$; for images the encoding $\phi$ is too hard to handcraft, so the network learns encoding and prediction together, which is why it needs more data. Success factors: data, GPUs, large models that are easier to train with simple gradient methods, and flexible "Lego" building blocks (CNN for images, RNN for text, GNN for molecules).
> - *Representations.* A unit is a linear classifier and a pattern detector. Two units in parallel map a red and blue dataset that no line separates into new coordinates $(0, 1)$, $(1, 1)$, $(1, 0)$ where one line separates it: "this is the power of deep learning, it internally creates new representations." Tanh and ReLU keep each point's distance to the boundary instead of collapsing clusters, which is one academic reason step functions are not used in hidden layers. Ten hidden units give a boundary made of straight segments; two would suffice in theory but the non convex optimization needs more; hundreds give smoother boundaries and overfitting. Layers build edges, then parts, then objects. Ten softmax outputs for the ten digits.
> - *Training.* Squared loss for regression, negative log likelihood for classification; the partial derivative of the loss with respect to a weight decides whether to increase or decrease it; stochastic gradient descent picks one point (or a batch) per update and shuffles per epoch; backpropagation (Rumelhart, Hinton and Williams, 1986) is the chain rule with memorization, a special case of automatic differentiation; non convexity means different initializations reach different minima; step sizes that are too small crawl and too large overshoot; AdaGrad, momentum and Adam; regularization by weight decay, early stopping, data augmentation, dropout (Srivastava et al., 2014) and batch normalization; Bengio's "Practical Recommendations for Gradient Based Training of Deep Architectures" as the reference.
> - *Case study.* Fashion MNIST: 60,000 training and 10,000 test images of 28 by 28 pixels in ten clothing classes, flattened into 784 inputs with categorical cross entropy.
>
> **Mentored session, hands on session and practice cases.** The mentor quiz covered layer types, the role of the activation function (a switch that adds non linearity; without it the network is one neuron), forward propagation, backpropagation, gradient descent, why deep networks overfit, batch normalization, dropout and early stopping. The case was **Fashion MNIST** for ElegantYou (785 columns, 6,000 images per class, so accuracy is a fair metric): decision tree 46.19 percent train and 45.98 test; random forest with 100 trees 73.15 and 72.34; an ANN with three hidden layers of 32, 64 and 32 ReLU units, 10 epochs, cross entropy and L1 regularization $10^{-5}$ reached 91.89 percent train and 87.87 test, confusing pullovers with shirts and coats. The hands on KNIME session (your timestamped summary of March 23) built the same comparison: Excel Reader, Value Counter, Statistics, z score Normalizer fitted on the training partition and applied to the test partition with Normalizer

Apply ("a common interview question": normalize after splitting to avoid leakage), a 70/30 Table Partitioner, Decision Tree Learner (gain ratio, MDL pruning), Random Forest Learner (depth 10), MLP Learner (200 iterations, two hidden layers, 10 then 32 neurons), six Scorers, an Expression node to tag each result with the model name, Row Filter on the "overall" row, Column Filter, Concatenate and two Bar Charts. Results: tree about 85 percent train and 79 test, forest about 70 percent on both (Java heap errors on some machines; the fix is the memory setting in knime.ini), MLP about 92 percent train and 85.2 test, the preferred model. The **Insurance Charges** KNIME practice case (1,338 policies; smokers' median charge USD 34,456 against USD 7,345; southeast the most expensive region): linear regression $R^2$ 0.76 train and 0.72 test with RMSE USD 6,034; RProp MLP with default settings $R^2$ 0.822 on test, and 0.831 with 100 iterations, 3 hidden layers of 15 neurons; predictions denormalized with the Denormalizer node gave RMSE USD 4,649. An extra case on **University Admission** (GRE, TOEFL, rating, statement, recommendation, GPA, research, chance of admission) was provided for RapidMiner.

**Your notes.** Your March 19 notes summarize every video with timestamps (two off topic "Unboxing" summaries slipped in), and you kept a 133 question condensed English version of the official FAQ under the French title "FAQ Réseaux de Neurones".

## Module 8. Computer vision methods

**What it taught.** Images as tensors, convolution as learned local filtering, pooling, the architecture of a convolutional neural network, transfer learning from pretrained networks, and image classification and object detection use cases. The no code demonstration used Google's Teachable Machine to train an image classifier from a webcam or uploaded images in minutes.

**The mathematics under it.**

A convolutional layer computes $(K * I)(i,j) = \sum_{a,b} K(a,b)\, I(i+a, j+b)$ for each learned kernel $K$; the kernel is shared across positions, which gives translation equivariance and far fewer parameters than a dense layer. Max pooling downsamples by taking the maximum over a window. Stacking convolution, nonlinearity, and pooling builds features from edges to textures to parts to objects. Transfer learning freezes the early layers of a network trained on a large dataset and retrains only the final layers on your images; that is what Teachable Machine does behind the interface, which is why it works with a few dozen examples.

*Sizes and parameter counts, as the handouts gave them.* An input of $n \times n$ convolved with an $f \times f$ filter, padding $p$ and stride $s$ gives an output of side $\lfloor (n + 2p - f)/s \rfloor + 1$; "same" padding uses $p = (f-1)/2$. A $6 \times 6 \times 3$ image feeding 10 dense units needs $108 \times 10 + 10 = 1{,}090$ parameters, while 10 filters of $3 \times 3 \times 3$ need $270 + 10 = 280$ whatever the image size. The vertical edge detector is the $3 \times 3$ filter with columns $1, 0, -1$; the horizontal one has rows $1, 0, -1$; Prewitt, Sobel (center weights $\pm 2$) and Laplacian (second derivative) filters are the classical hand designed versions of what the first layer learns. Max pooling has no learnable parameters and is applied per channel.

**In the tools.** Teachable Machine (browser), with export of the trained model, and KNIME's image processing and Keras nodes for a full pipeline. (Correction: the module's tool content was Teachable Machine only, in image mode for the rice case and in pose mode for the yoga case; no KNIME image pipeline was delivered. Teachable Machine reserves 15 percent of the uploaded images for validation and shows per class accuracy, the confusion matrix and the accuracy and loss curves under "Under the hood".)

**Case studies.** Defect detection in manufacturing, product recognition in retail, and medical imaging as an example where responsible AI questions appear (Module 12). (The delivered cases were rice grain classification, yoga pose detection, and Fashion MNIST with a CNN.)

**Keep.** Data quality and class balance matter more than architecture at your scale. Look at the misclassified images; they usually reveal a labeling or lighting problem, not a model problem.

> **What the MIT lectures actually said.** Fifteen videos of about 90 minutes: why dense networks fail on images (a $100 \times 100$ image flattened gives millions of parameters and loses the 2D layout); spatial locality and translational invariance, obtained by sliding a local detector (convolution), sharing its weights, and pooling; 1D convolution as pattern matching with stride and padding; 2D and 3D convolution with filter banks (a horizontal and a vertical edge detector in layer 1 combine into a box detector in layer 2); the hierarchy edges, shapes, parts, objects, scenes (AlexNet visualizations); the Fashion MNIST CNN (two convolution layers of 16 filters of $3 \times 3$, $2 \times 2$ max pooling, flatten, two dense layers with batch normalization, softmax, Adam for 10 epochs with mini batches of 128) reaching 91 percent against 87 percent for the dense network and fixing the coat versus pullover confusion; depth and ResNet skip connections; CNNs are translation invariant but not rotation invariant (accuracy falls with the rotation angle except for square objects at multiples of 90 degrees), so augment with rotated images; algorithmic bias (commercial gender classifiers worst on darker skinned women) and shortcut learning (cars recognized by the sky or the road, a pneumonia model that only worked in its own hospital, a panda turned into a gibbon by imperceptible noise), with data augmentation and pretraining as remedies.
>
> **Mentored session (March 28) and case study.** The quiz: ANNs cannot capture spatial relationships; convolutions apply filters; pooling reduces dimensions; max pooling keeps the most prominent feature; augmentation improves generalization; stride reduces computation; padding preserves the edges. The case was **rice type classification** for the Department of Agriculture (5 types, Arborio, Basmati, Ipsala, Jasmine, Karacadag; 4,063 training images of about 810 per class on a black background). In Teachable Machine the model reached about 98 percent after 5 epochs with minimal Basmati versus Jasmine confusion. Your Granola summary of the session adds the mentor's own points: a phone camera at $720 \times 1080$ gives 720,000 features and you would need about ten observations per feature, which is why CNNs exist; the industry workflow is "existing model, your data, customized model" (Faster R CNN trained on COCO detects 90 object types; keep the feature layers, replace the classifier); a model forced to choose among five rice types classified a photo of a politician as 64 percent Basmati, so production models need an "other" class; the mentor's own car damage project (2017 to 2018) replaced a 3 to 5 day surveyor visit by a 3 minute assessment, with manual bounding boxes for dents and scratches, augmentation with parking lot photos, and human review below 92 percent confidence; stop training when test loss starts rising; the wolf versus husky model that was actually detecting snow.

**Your own case study: Yoga Pose Detection.** This is the piece of work in the archive that goes furthest beyond the brief. You built it as "Gil Paul Pelletier, Data Science Manager, Great Fitness" (with "MIT Data Analysis, UMontréal Mathematics" as the credentials line) in two versions, a 10 slide deck and a 13 slide extended deck.

- *Problem.* The app's recorded yoga tutorials cannot tell users whether they hold a pose correctly; classify a photo into one of five poses (downdog, goddess, plank, tree, warrior) and use the confidence as a quality score; a cheap v1 to validate demand.
- *Method.* Teachable Machine in Pose Project mode: a pretrained pose estimator reduces the body to limb vectors and joint angles, and the classifier learns the geometry, ignoring background and clothing. Your analogy: "read the mechanism, not the headline."
- *Data.* 1,081 training images (plank 266, warrior 252, downdog 223, goddess 180, tree 160), acceptable imbalance; 60 held out test images, 12 per pose, never seen in training; internal 85/15 split.
- *Training.* 50 epochs, batch 16, learning rate 0.0001, all defaults kept; raising the learning rate overshot and cost several classes accuracy ("small step, capped downside"). Learning curves: training accuracy 0.57, 0.90, 0.94, 0.96, 0.98 at epochs 0, 5, 10, 15, 20 and 0.99 from epoch 30; validation accuracy 0.62, 0.83, 0.85, 0.84, 0.87 and about 0.85 to 0.87 thereafter; validation loss bottomed near epoch 20 at 0.58 then drifted to 0.80, "the last 30 epochs tightened memorization without improving generalization."
- *Per class validation accuracy.* Tree 0.96, downdog 0.91, plank 0.90, goddess 0.85, warrior 0.68. The confusion matrix shows warrior leaking evenly into goddess, plank and tree (4 each) because its wide stance, bent knee and extended arms sit near several decision boundaries at once.
- *Held out test.* 100 percent on every class, including warrior; your explanation is that the test photos are clean and well framed, so real webcam input will land between the two numbers.
- *Recommendations.* Pre process every image on the backend (resolution, crop, lighting) so the feature works on a budget phone; add warrior images near its confusion boundaries ("targeted data beats more epochs"); add poses in batches and re read the confusion matrix each time; move to a custom CNN by transfer learning if the feature gains traction.
- *Extension.* You classified the same 60 test images zero shot with Mistral's Pixtral, a 12 billion parameter vision language model, from a one sentence instruction. Overall Teachable Machine about 86 percent (validation) against Pixtral 80 percent; Pixtral scored 100 percent on tree, goddess and plank and 50 percent on downdog and warrior. Downdog versus plank is where the pose model wins because it reads the hip angle; warrior versus goddess is confused by both models, so "the confusion lives in the poses, not the model." Your three conclusions: structure beats size sometimes; identical failures across architectures point to the data; this mirrors Mistral's own finding that Pixtral needed fine tuning on satellite imagery. You flagged that 60 images is a small test and that the two setups are not a matched benchmark. A French chart ("Même tâche, deux philosophies, où chacune échoue") and an English one, plus two LinkedIn cards dated June 1, complete the folder.

**Your notes.** The "Notes Computer Vision Methods" file is the course reader (outline, CNN handouts, ANN drawbacks handout, lecture notes) stamped with your watermark; your own study guide printed on March 28

> restates the twelve lectures and recommends three videos (IBM Technology on CNNs, 3Blue1Brown "But what is a convolution?", and MIT 6.S191 by Alexander Amini). A 95 question CNN FAQ from Great Learning is also in the folder.

## Module 9. Project week: prediction methods with deep learning and neural networks (Project 6)

**The task.** A graded project in which you built a neural network model on a provided business dataset, compared it with the classical models of earlier modules, and reported performance and recommendations. The project was due on April 20, 2026 at 12:30 EST. Optional support sessions were held on April 11 (RapidMiner flow) and April 12 (KNIME flow); you were told to join only the session for the tool you used.

**Structure that the graders expected** (from the project guide the office referred to):

1. Problem statement and business objective in your own words.
2. Data description and exploratory findings.
3. Preprocessing decisions with justification.
4. Model building, at least one neural network and one comparison model, with hyperparameters stated.
5. Evaluation on the test split with the right metrics.
6. Business insights and recommendations, plus limitations.

> **The actual brief.** Available from March 12, due April 20, 2026, 40 marks, PDF or PowerPoint upload. A bank's consumer credit department wants an empirically derived, statistically sound and interpretable credit scoring model for home equity lines of credit, in line with the Equal Credit Opportunity Act, that predicts which applicants will default and identifies the features that matter. The **HMEQ** dataset has 5,960 loans, the target BAD (1 for default or serious delinquency, 1,189 cases, 19.95 percent) and twelve inputs: LOAN, MORTDUE, VALUE, REASON (home improvement or debt consolidation), JOB, YOJ (years on the job), DEROG (major derogatory reports), DELINQ (delinquent credit lines), CLAGE (age of the oldest credit line in months), NINQ (recent inquiries), CLNO (credit lines) and DEBTINC (debt to income ratio). Eleven of the thirteen columns have missing values, worst DEBTINC (21.3 percent) and DEROG (11.9 percent). Rubric: problem and EDA 8, model without tuning 10, model with tuning 10, best model and metric 6, insights 6. The Business Interpretation template carried the reference numbers of the teacher's KNIME workflow: basic network accuracy 84.94 and kappa 0.327, tuned 88.70 and 0.596, optimized 89.87 and 0.633.
>
> **How you built it.** You took the hands on track in KNIME 5.9.0 on macOS. On March 21 you worked through the teacher's workflow with an AI assistant in French (the 50 page chat is in the folder): CSV Reader, Column Filter, Missing Value (mean for MORTDUE, VALUE, YOJ and DEBTINC, fixed 0 for DEROG, median and most frequent in your variant), Category to Number for REASON and JOB, a Rule Engine creating the nominal target BAD_nom ("Bad" if BAD = 1, else "Good") because the RProp MLP Learner needs a nominal class, a Column Filter dropping the numeric BAD, a Min Max Normalizer, a Table Partitioner at 70 percent stratified on

BAD_nom with seed 42, then Linear Correlation, Statistics and Bar Chart for EDA, the base RProp MLP Learner (the teacher's setting was 10 hidden layers of 10 neurons and 200 iterations), the Parameter Optimization Loop with a Table Creator grid, and the final learner with the best parameters. The chat also records the two errors that cost you an hour: a warning on the Parameter Optimization Loop Start because the KNIME Optimization extension was not installed (File, Install Extensions, search "optimization"), and an RProp MLP Learner error "Value (=0.0) must be within the range [1...1000000]" caused by a flow variable inherited from the tuning loop, fixed by removing the flow variable connection. You ended that session by re importing the teacher's workflow and running it whole (Shift F7).

**What you submitted** (18 slide deck "HMEQ Loan Default Prediction, MLP Neural Network, KNIME Platform", exported March 28 at 05:14, plus two written reports and two dashboards). The pipeline on the deck: preprocessing, EDA, 70/30 stratified split, base MLP, **SMOTE** oversampling (rate 2, K = 5, applied to the training port only), a refined 770 configuration grid, and the final model.

*EDA.* Median loan USD 16,300 (range 1,100 to 89,900), average debt to income 33.8 percent; 78 percent of borrowers have zero delinquencies. Default rate by DELINQ: 0 gives 14 percent, 1 gives 34, 2 gives 45, 3 gives 55, 5 gives 82, 6 or more gives 100. By job: Sales 34.9 percent, Self employed 30.1, Manager 23.3, Other 23.2, Professional 16.6, Office 13.2. By DEBTINC quartile: 5.5, 5.9, 7.4 and 15.5 percent. Correlations with BAD: DELINQ $+0.347$, DEROG $+0.270$, NINQ $+0.170$, DEBTINC $+0.155$, CLAGE $-0.166$, LOAN $-0.075$.

*Models.*

| Model | Configuration | Accuracy | Precision (Bad) | Recall (Bad) | F1 (Bad) | Kappa | TP / FN / FP / TN |
|---|---|---|---|---|---|---|---|
| Base MLP | KNIME defaults, imbalanced data | 82.9% | 66.2% | 29.7% | 0.41 | 0.327 | 106 / 251 / 54 / 1,377 |
| Teacher's tuned MLP | 880 configuration grid; topology 2, 45 neurons, 200 iterations, learning rate 0.001 | 89.9% | 88.9% | 56.3% | 0.690 | 0.633 | 201 / 156 / 25 / 1,406 |

| Model | Configuration | Accuracy | Precision (Bad) | Recall (Bad) | F1 (Bad) | Kappa | TP / FN / FP / TN |
|---|---|---|---|---|---|---|---|
| Your improved MLP | SMOTE rate 2, K 5 on training only; 770 configuration grid; topology 2, 49 neurons, 300 iterations, learning rate 0.001 | 90.3% | 84.3% | 63.0% | 0.721 | 0.664 | 225 / 132 / 42 / 1,389 |

The teacher's grid was topology 1 to 2, neurons 5 to 50 by 5, iterations 50 to 200 by 50, learning rate 0.001 to 0.1 by 0.01, that is $2 \times 10 \times 4 \times 11 = 880$ runs with Cohen's kappa as the objective. Your grid fixed the topology at 2 and searched neurons 35 to 55 by 2, iterations 150 to 300 by 25 and learning rate 0.0005 to 0.005 by 0.0005, that is $1 \times 11 \times 7 \times 10 = 770$ runs, because the teacher's optimum sat at the edge of its ranges. SMOTE moved the training class ratio from 20/80 to about 33/67 (about 834 to about 1,668 Bad rows). Against the teacher's model you caught 24 more of the 357 test defaults at the cost of 17 more false alarms, "1.4 defaults caught per false alarm added", with precision down 4.7 points and specificity at 97.1 percent.

*Metric choice and recommendations.* Recall for the Bad class as the key metric (a missed default is a loss, a false alarm is a manual review), F1 as the secondary. Recommendations: deploy the SMOTE tuned MLP as the primary screen; lower the decision threshold from 0.50 to 0.35 to reach about 75 percent recall; a two tier review (auto approve low risk, humans for the rest); automatic manual review for DELINQ of 3 or more and top quartile DEBTINC; quarterly retraining; an ensemble with a decision tree for explainability; estimated savings of about USD 447,000 per batch from the 24 extra defaults caught at the average loan of USD 18,608. Limitations you listed: recall still 63 percent, a black box, a single split with no cross validation, and synthetic SMOTE rows that may not represent real defaults.

*A discrepancy to know about.* The KNIME screenshots you assembled show the base model scorer at accuracy 84.1 percent, kappa 0.385, recall 34.7 percent and precision 70.9 percent (TP 124, FN 233, FP 51, TN 1,380), while the deck and the reports use 82.9 percent, 0.327 and 29.7 percent (TP 106, FN 251, FP 54, TN 1,377 or 1,380). The improved model's figures are identical everywhere. The base figures in the deck match the template's kappa of 0.327, so they probably came from the teacher's reference run rather than from your own execution. See Appendix C. No grade or feedback exists in the folder.

## Module 10. Generative AI foundations

**What it taught.** What a large language model is, how it is trained (next token prediction on very large corpora, then instruction tuning and reinforcement from human feedback), tokens and context windows, temperature,

hallucination, prompt engineering (role, task, context, format, examples, chain of thought), and the main tools used without code: ChatGPT, Gemini, NotebookLM for grounded question answering on your own documents, and DALL E for images. Your mentored session was on Saturday April 18, 2026 and the course assessment was due April 20.

**The mathematics under it.**

A language model factorizes $p(w_1, \ldots, w_T) = \prod_t p(w_t \mid w_{<t})$ and is trained by minimizing cross entropy of the next token. The transformer computes each layer with self attention: for queries, keys, and values $Q, K, V$ obtained by linear maps of the token embeddings,

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

so each token forms a weighted average of the values of all previous tokens with weights that depend on content. Sampling with temperature $\tau$ uses $p_i \propto \exp(z_i/\tau)$; $\tau \to 0$ is greedy decoding, higher $\tau$ increases diversity and error. Diffusion models for images learn to reverse a gradual noising process, which is why generation starts from noise.

> *Dahleh's way of presenting the transformer.* Language has syntax, so the relation between tokens is non linear; embed each token in a high dimensional vector $x_i$ so that the relation becomes a linear operator. Attention from token $j$ to token $i$ is $A_{ij} = \langle x_i,\ K_1 x_j + K_2 s_{ij} \rangle$ with $s_{ij}$ the position relation; the embeddings are updated iteratively, $x_i(t+1) = x_i(t) + \sum_j w_{ij} V x_j$ with $w_{ij} = e^{A_{ij}} / \sum_k e^{A_{ik}}$, which clusters them, and each iteration is one layer; the matrices $(K_1, K_2, V)$ are learned by supervised next word prediction on hidden words. The sparse data view: with a 50,000 word vocabulary and 100 word prompts there are $50000^{100}$ possible contexts, so next word prediction is a matrix completion problem in an unimaginably sparse frequency matrix, and low rank methods that worked for Netflix do not suffice because syntax is non linear. The supplementary videos add induction heads (layer 1 copies the previous token by position, layer 2 finds the same content and copies its successor, which is how "Harry" predicts "Potter" and how in context learning works), multi head attention, causal masking, sinusoidal, ALiBi and rotary positional encodings, residual connections, the quadratic cost of attention, mixture of experts, and transformers on images (patches) and graphs (nodes as tokens, eigenvector positional encodings).

**Keep.** A prompt is a specification. State role, goal, inputs, constraints, and output format, give one or two examples, and ask the model to show its reasoning when the task is multi step. Verify every fact the model states; fluency is not evidence.

> **What the MIT lectures actually said.** Six videos and five content pages by Munther Dahleh: generative models are not new (a fitted density is a generative model; a random forest generates labels; an AR(p) process generates the future; matrix completion generates missing ratings; AlphaFold generates protein structures); what is new is generation from sparse data; autoencoders and diffusion models (forward noising as a dynamical system whose inverse is learned as a model of the noise); LLMs as next word predictors that were never optimized for factual truth; hallucination illustrated with the professor's own name (a model invented a "Prof. Yaser S. (Yazeed)

Dahleh" and a Davidson College basketball player); RLHF and the question "whose values"; data rights (the Google, OpenAI and New York Times disputes), safety, accountability; tuning versus training (soft prompts trained with frozen weights, hours to days, versus pretraining on trillions of tokens for weeks); agentic AI defined by goals, autonomy, perception, planning and adaptation with the self driving car as the reference system; and the ACT.org college assistant demo. The prompt engineering pages taught system, user and assistant messages; zero shot, few shot and chain of thought; temperature 0 to 0.3 for factual work and 0.7 to 1.0 for creative work; max tokens, top p and frequency penalty; the draft, run, observe, refine loop; JSON output constraints; guardrails such as "if unsure, respond I don't know"; and an evaluation checklist (consistency, relevance, structure, factuality, clarity, efficiency) run on 10 to 15 test cases including edge cases.

**Mentored session (April 18): an agentic web search assistant in n8n.** The case was MarketPulse Intelligence. You built, in the Great Learning n8n lab, a main workflow (Chat Trigger, AI Agent, OpenAI Chat Model, Simple Memory with a window of about 10 messages) and a "Search web" sub workflow (Execute Sub workflow trigger with inputs SearchQuery and MaxSitesToSearch, a Wait node of 5 seconds because DuckDuckGo fails when called too fast, the DuckDuckGo community node with safe search off, an Edit Fields node keeping only description and url, and an Aggregate node returning two arrays of 10), published it, attached it to the agent as "Call n8n Workflow as Tool", wrote the research assistant system message (always search before answering, read all snippets, synthesize across sources, cite each page with its URL, never invent, stay neutral, the current date injected as an expression), and added an evaluation node (OpenAI "Message a Model" with GPT 4.1 Mini) that reproduces the answer and grades Relevance, Grounding, Completeness and Clarity as Pass or Fail. This session is the template that you reused, with your own choices, in the MediSure project.

**Practice exercise: Browbake.** A bakery needs promotional content. Task 1, a poster in Google Gemini with a simple prompt and then an improved one (a system message casting the model as a food brand designer, an Instagram composition, brand name in the top third, tagline "Pure Bliss in Every Bite", a caramel, brown and cream palette, a 120 pixel area for contact icons). Task 2, an eight second video in Adobe Firefly (Veo 3.1 model) from a storyboard prompt. Task 3, a voiceover in ElevenLabs ("Experience pure indulgence with every bite..."). Task 4, merge in Qooly. Your outputs of April 15: three ElevenLabs takes (two with the Charlotte voice, one with Jeanne Mance), the Firefly video, and merged videos for the brownie brief plus your own chocolate cake and carrot cake variants. You then wrote a single page web application, "Browbake AI Video Creator", with a five step progress bar, a campaign form that fills the brand, products, tagline, call to action, palette, contact and script into every prompt, simple and improved prompt tabs with copy buttons, direct links and recommended settings for each tool (Firefly at 720p, 16:9, 24 fps, 8 seconds, audio off; ElevenLabs Charlotte on Eleven Multilingual v2, clarity high), checklists, and local storage of progress. One tip mentions "Click Download (Télécharger)", a trace of your French language Firefly interface.

**Your notes.** Your April 19 export of the module pages includes the transformer supplement and the session plan; Dahleh's own deck is in the folder.

## Module 11. Business applications of generative AI

**What it taught.** Moving from chat to systems: retrieval augmented generation (RAG) so the model answers from your documents, agentic workflows in which a model plans and calls tools, evaluation of generative outputs, cost and latency, and the typical business applications (customer support, document processing, marketing content, code assistance, internal knowledge search). This module carried a quiz (due April 27, 2026) and a course project (due May 4, 2026). A project support session on "Gen AI and Applications" was held on April 26.

**The mathematics under it.**

RAG turns documents into chunks, embeds each chunk into a vector $e \in \mathbb{R}^d$, and at query time retrieves the chunks whose embeddings have the highest cosine similarity $\cos(q, e) = q^\top e / (\|q\| \|e\|)$ with the query embedding; those chunks are placed in the prompt as context. Quality depends on chunk size, overlap, the embedding model, and the number of retrieved chunks $k$, and it is measured with retrieval recall and with answer faithfulness (does the answer stay inside the retrieved text). An agent is a loop: observe, plan, choose a tool, act, observe again, until a stopping condition; the model's output at each step is parsed into a tool call.

**In the tools.** NotebookLM and custom GPTs as no code RAG, plus workflow builders for chaining prompts and tools. The project asked for a concrete business use case, a designed prompt or workflow, sample outputs, an evaluation, and a discussion of risks. (Correction: the tools were Google AI Studio with Gemini 2.5 Pro for the classification and summarization cases, NotebookLM for the HR policy case, and n8n with OpenAI models, Pinecone or Weaviate and DuckDuckGo for the mentored session and the project. The project brief was fixed, not free: the MediSure regulatory intelligence assistant described below.)

**Keep.** The retrieval step is where most RAG failures happen. Inspect what was retrieved before blaming the model.

> **What the MIT lectures actually said.** Three videos and two content pages: companies choose between adopting a pretrained model and tuning it for tasks, or training their own model on their own data (a bank's customers, regulations and products), which the Mosaic ML pricing shows to be affordable for small models; transformers apply beyond language wherever tokenization works (genes, images); text to label tasks (support triage, product categorization, sentiment, moderation, intent) and text to text tasks (executive summaries, marketing copy, report digests); the classification workflow (define distinct labels, gather 50 to 200 examples, write a strict system prompt with few shot examples and a "needs human review" fallback, measure accuracy, precision, recall and F1, iterate); the generation workflow evaluated by an LLM as a judge, human review, ROUGE for summaries, BLEU for rewrites, and a faithfulness check; when prompting fails (private documents, very long sources, near perfect consistency, facts after the training cutoff) use RAG: chunk, embed, store in a vector database, retrieve the top 5 to 10 chunks, optionally re rank, inject them with instructions to use only the sources, answer with citations or "insufficient information", log and collect feedback; fine tune only for precise repeatable behavior at volume, and often combine (RAG for facts, a tuned model for tone); agentic AI as a two layer architecture in which an orchestrator plans and calls tool models (verify the customer, check the order, check the policy, refund, update); guardrails today are mostly human feedback and fine tuning, which is limited, and The Economist's finding that the rush to agents has not yet paid off.
>
> **Mentored session (April 25): agentic RAG for NovaMind AI.** A weekly pipeline in n8n: Schedule Trigger every 7 days, date arithmetic for the last week, an HTTP Request to the arXiv API for the newest cs.LG and stat.ML

papers, XML to JSON, Split Out, a Set node preparing id, title, summary, authors, date and categories, Remove Duplicates, an enrichment AI Agent with a Structured Output Parser that assigns a primary category (from a taxonomy of 14 including PEFT, RAG, quantization, agentic AI, multimodality), up to two secondary categories and an impact score from 1 to 5, a Merge, then a Weaviate Vector Store insert with OpenAI `text-embedding-3-small` embeddings and a recursive splitter of 2,000 characters. Part 2, triggered by a static session id once the upload is confirmed, runs an AI Agent that must query the Weaviate store as a tool, aggregate counts by topic, pick 2 to 3 trends with 1 or 2 representative papers each, and return strict JSON with a subject line starting "NovaMind AI Weekly Research Update:" and a Markdown body with clickable arXiv links, converted to HTML and sent by AWS SES. The rules "always retrieve before reasoning, always aggregate before trend identification, never hallucinate" appear verbatim in the system message.

**Practice cases** (no written answers of yours are stored): hotel guest review classification in Google AI Studio with a strict six label system prompt and few shot pairs; patient friendly summaries of 25 clinical notes in a fixed three part format at temperature 0.2 to 0.4; an HR policy assistant in NotebookLM over an employee handbook, verified by clicking the citation numbers.

**Your project: the MediSure Regulatory Intelligence Workflow.** Brief (available April 9, due May 4, 40 marks): MediSure Devices, a mid sized medical device manufacturer, must monitor FDA, EMA, CDSCO and WHO announcements. Build an agentic assistant in n8n that answers natural language compliance questions from a corpus of four PDFs (the FDA Patient Engagement Advisory Committee executive summary on recall communications of October 2021, the FDA Section 522 postmarket surveillance guidance of October 2022, the European Commission's December 16, 2025 questions and answers on simpler medical device rules, and Commissioner Dombrovskis's remarks of the same day) and from web search, produces a structured briefing (key changes, affected device categories, effective dates, required actions), and evaluates its own answers. Rubric: Workflow 1 vector indexing 13, Workflow 2 retrieval agent 7, Workflow 3 web search sub workflow 14, Workflow 4 evaluation 5, screenshot clarity 1.

You planned it on April 27 (10 to 13 hours over 8 days, a risk register, screenshot naming conventions, open decisions on chunking, model and search provider), wrote a Pinecone setup recipe the same day (the index dimension must match the embedding model exactly and cannot be changed later, 1,536 for `text-embedding-3-small` and 3,072 for `text-embedding-3-large`; tick "custom settings" or Pinecone will host its own embeddings; lowercase names with hyphens only; the key is shown once), and built everything on your own n8n instance on Hostinger rather than in the Great Learning lab. The final PDF (78 pages, exported May 2 at 04:06, title slide "Gilbert Paul Pelletier, Business Applications of Generative AI, Great Learning, May 1, 2026") follows the official template step by step with a screenshot and a rationale per step.

- *Workflow 1, indexing.* Manual Trigger; instead of uploading files to n8n's disk you published the four PDFs in a public GitHub repository (`woodlandbay/medisure-regulatory-docs`) and fetched them with a Code node listing the raw URLs and an HTTP Request in binary mode ("auditable, reproducible, no infrastructure dependency"); a small Code node rewrote the MIME type from octet stream to PDF; Pinecone Vector Store in insert mode with the credential "Pinecone MediSure", index `medisure-regulatory-index` (dense, 3,072 dimensions, cosine, serverless on AWS us east 1, free Starter tier); OpenAI embeddings

`text-embedding-3-large` with dimensions explicitly set to 3,072 because n8n's default of 1,536 would fail; Default Data Loader in PDF mode with the filename attached as metadata for citations; Recursive Character Text Splitter with chunk size 1,000 and overlap 200 ("about 250 tokens surfaces the specific clause while keeping 2 to 6 sentences of context; 20 percent overlap is the standard ratio"). Result: 224 chunks in two batched embedding calls, verified in the Pinecone console.

- *Workflow 2, retrieval agent.* Chat Trigger (session id and chat input); AI Agent; system message built on seven principles (role and audience, tool inventory, mandatory retrieval before answering, a citation format "Source: filename.pdf", strict FDA versus EU separation, a five section briefing structure mirroring the brief, uncertainty flagging and a scope refusal); chat model `gpt-5-mini` through the Responses API (a temperature of 0.3 was rejected by the model as unsupported and removed); Simple Memory with a window of 10; the Pinecone store as a tool named `medisure_regulatory_search` with a limit of 5 and metadata on; the same embedding model and dimension for retrieval, which you called "the central correctness invariant of RAG".
- *Workflow 3, web search.* A separate sub workflow with a typed contract (SearchQuery as string, MaxSitesToSearch as any type because the model sometimes sends numbers as strings), the DuckDuckGo community node (no API key, no cost, worldwide region, safe search off so that "implant" or "pediatric" are not filtered), Edit Fields to keep description and url (about 300 tokens instead of more than 1,000), Aggregate into two arrays, a 5 second Wait, published as version 1.0. Attached to the agent as `web_search` with routing rules (web for recent developments, Pinecone for Section 522, PEAC, EU MDR timelines and the Dombrovskis remarks).
- *Workflow 4, evaluation.* A duplicate of Workflow 2 with a Basic LLM Chain hosting a `gpt-4.1` judge, deliberately a different model family from the agent; a four dimension rubric (Relevance, Grounding, Completeness, Clarity) scored 1 to 5 with a PASS, PARTIAL or FAIL verdict in strict JSON, with the rule that a correct refusal or an honest search failure counts as correct behavior; Edit Fields producing user query, agent response and judge evaluation.

*Tests you documented* (from your test notes of April 28 and 29): Test 1, "What does FDA Section 522 require for postmarket surveillance studies, and which devices does it apply to?" produced the five section briefing with every paragraph cited, the four statutory triggering categories, a cross reference to 21 CFR 822.7 and the enforcement consequences (misbranding, seizure, injunction, civil penalties), using only Pinecone. Test 2, FDA versus EU philosophies, kept the two jurisdictions under separate headers, cited three PDFs, identified "least burdensome" on the FDA side and "proportionality", "digitalisation" and real world evidence on the EU side, noted that FDA guidance is non binding and the EU texts are proposals, and offered to escalate to web search. Test 3, the most recent 2026 FDA guidance on AI enabled devices, called the web tool seven times with reformulated queries (about 25,000 tokens, 13.8 seconds), got unrelated results, and refused to fabricate: "I could not retrieve a verifiable 2026 FDA guidance document." Test 4, a 401(k) retirement question: the first version of the prompt, after the web tool was added, answered with about 400 words of investment advice (equity allocation, the 4 percent rule, Roth conversions), a failure you diagnosed as the scope principle being pushed down the attention budget by the longer prompt; version 2 named examples including 401(k), added "decline immediately without calling any tool" and a fixed template, and the retest produced a 50 word refusal with zero tool calls. The final run

> scored 5 on every dimension for all four tests (tokens 17,842; about 25,000; 158,834 for an in corpus De Novo versus 510(k) comparison that replaced the unstable web test; 1,094 for the refusal).
>
> *Lessons you wrote down.* The Edit Fields node had stored $json.description`as literal text instead of an expression, so the agent received the strings"
> `json.description" and "$json.url"; standalone tests had passed because nobody inspected the array contents, and the agent's uncertainty flag caught it; the fix was to drag fields from the input panel and republish as version 1.1. Live web queries are diagnostics, not regression tests, because DuckDuckGo results vary from day to day. Choosing different model families for the agent and the judge reduces shared blind spots. Your plan had foreseen` gpt-4o`, a paid search API and n8n's disk reader; the build used` gpt-5-mini`, DuckDuckGo and GitHub raw URLs. No grade or feedback exists in the folder.

## Module 12. Ethical and responsible AI

**What it taught.** Bias and fairness, transparency and explainability, privacy, accountability, safety, and governance frameworks (the EU AI Act, the NIST AI risk management framework, and internal model risk practices). The course assessment was due May 4, 2026.

Correction: neither the EU AI Act nor the NIST framework appears anywhere in the module. The delivered content is Munther Dahleh's seven lectures from his book *Data, Systems, and Society: Harnessing AI for Societal Good* (Cambridge University Press): the AI lifecycle, bias and its examples, racial bias and statistical parity, causality, privacy, interconnections and domains, and interdependency and feedback in AI systems. Governance was treated through GDPR, the US antitrust debate and a four way framework (law, architecture, social norms, markets) attributed to Lessig.

**The mathematics under it.** Fairness definitions are constraints on error rates across groups $g$: demographic parity asks $\Pr(\hat{y} = 1 \mid g)$ to be equal across groups, equalized odds asks true and false positive rates to be equal across groups, and calibration asks $\Pr(y = 1 \mid \hat{p}, g) = \hat{p}$ for every group. An impossibility result states that, when base rates differ across groups, calibration and equalized odds cannot both hold, so fairness is a choice, not a computation. Explainability tools such as feature importance and SHAP values (Shapley values from cooperative game theory applied to features) were shown inside the tools. (Correction: SHAP was not shown; the module's explainability content was the requirement that a decision cite the evidence that drove it.)

> *The course's own definitions.* **Statistical parity difference**, the metric of the mentor case, is $\text{SPD} = \Pr(\text{approved} \mid \text{group}) - \Pr(\text{approved} \mid \text{overall})$, or between two groups $\Pr(\hat{y} = 1 \mid G_1) - \Pr(\hat{y} = 1 \mid G_2)$; values beyond $\pm 0.1$ were flagged. Example from the quiz: 80 percent of group A and 60 percent of group B approved gives $\text{SPD} = -0.20$ for B. **Counterfactual fairness** asks whether the decision would change if only a protected attribute were flipped, an individual level test; the lecture applies it to New York stop and frisk by building a model of how the arrests were generated and flipping race. The lecture's parity

discussion adds two warnings: a theorem says an algorithm cannot achieve parity on all metrics at once, so choose the metric you care about (in cancer diagnosis, false negatives); and if the feature distributions themselves differ between groups (missed credit card payments differ because affluent banks phone their customers first), balancing on that feature makes no sense, so mine for features with balanced distributions (rent history, education, family size, job stability). **Differential privacy**: the output of a computation with and without your record should be nearly the same; the median is not differentially private; approximate differential privacy works within error margins.

**Keep.** Document the data, the intended use, the metrics by group, and the human override. That documentation is what regulators and auditors ask for.

**What the MIT lectures actually said.** From your May 4 transcript (2 hours 8 minutes, the seven videos recorded back to back) and the module reader:

1. *The AI lifecycle.* Data (often unsynchronized, hard to curate) feeds an algorithm; the decision is a fusion of the algorithm's recommendation and a human or another model; the deployed decision feeds back into the data. Benefits: healthcare (X ray reading), infrastructure (renewable energy with pricing incentives), agentic systems (self driving cars), finance, and knowledge discovery (cancer, Alzheimer's). Four eras: mainframes (1950s: Turing, the perceptron, weather and air traffic), personal computers and ARPANET (1980s: distributed computing), the communication revolution (mid 1990s: phones and the web generate personal data, which "brought machine learning out of the slump"), embedded agent based systems (2005 onward). "AI is seventy years old, not born with LLMs in 2022; progress comes in step functions." Challenges are listed on the data side (sparse, high dimensional, noisy, confounded, privacy, rights), the algorithm side (efficiency, complexity, robustness, fairness), the decision side (fusion with humans, causality, ethics, economics) and the feedback side (sampling bias, confounding).
2. *Bias and its examples.* Speech recognizers from five vendors with much higher word error rates for Black speakers; a soap dispenser that ignores darker hands; STEM advertising delivered to men because retailers outbid STEM advertisers for women in the second price auction and the click data then "confirms" the perception (the auction is agnostic; the bias is a perception plus a feedback loop; the remedy is to reset the data); healthcare cost used as a proxy for health, which misreads underserved patients as healthy; the methodological dispute between Piketty and Auten and Splinter on US inequality, where the same public data gives a gap 30 percent smaller depending on the income unit, undeclared income and government transfers ("data science is not Python, data science is a science; publish your assumptions").
3. *Racial bias and statistical parity.* Census categories as self fulfilling social structures amplified by computation; equality versus equity (the Aristotelian "treat likes alike", the stools of different heights); removing the race column is not enough because zip code reconstructs it, and removing too much leaves no data; generative de biasing by synthesizing balanced records for résumé screening, with the difficulty of choosing the distribution; the parity metric choice above.
4. *Causality.* Ice cream and suntan are both caused by the sun; randomized controlled trials estimate the average treatment effect but not the personal one; observational causal inference (matching, learned confounders) for drug repurposing, where some hypertension drugs appear to accelerate dementia; the Netflix problem as

generation with missingness that is not at random (a viewer skips a genre on purpose); bandits and reinforcement learning reinforce confounding unless they explore; counterfactual fairness and the surveillance feedback loop (more cameras in Black neighborhoods, more arrests, more cameras).

5. *Privacy.* Phones, cars and workplaces leak enough to map a day; Europe and Korea regulate more than the US; Clearview AI scraped faces and its CEO tracked his daughter's boyfriend; Facebook and Cambridge Analytica (the book's careful statement that causal influence on the 2016 election remains hard to assess); the Golden State Killer caught through a relative's genome, an "externality" because your data is a node in a network, which GDPR's opt in and opt out categories miss; differential privacy; self driving cars and the trolley problem (surveys show people want a car that sacrifices its passenger but would not buy it, so the locally right answer raises deaths globally); deepfake calls from "your kid" and the inadequacy of an algorithm policing an algorithm.
6. *Interconnections and domains.* Atlanta, November 2014: two inches of snow, schools closed, everyone drove home at once, and the delays cascaded to a three hour delay in Los Angeles because pilots could not reach the Atlanta airport; bank runs as the financial analogue; low probability events live in the low data regime (rare diseases, the Texas gas freeze), so understanding the mechanism of the cascade matters more than data volume; high frequency trading turns any signal into large fluctuations; COVID policy as reinforcement learning (the early no mask advice was reasonable given knowledge and supply, then reversed; "revising decisions is exactly what science is"); the triangle of physical systems, social behavior and institutions with heterogeneous data as the glue, and the insistence that the same data must be abstracted according to the question asked.
7. *Interdependency and feedback.* The block diagram: current state of the world, perception (data, objective, model), algorithm development, then algorithm use repeated many times, each use producing an action that may or may not return to the data (opt in sampling bias). Five considerations: AI systems have intricate dependencies that entrench bias; they are always subjective; they offer opportunities for power and manipulation without detection; collective action must be modeled; they may have unanticipated objectives. Examples: Hinton's 2016 "we should stop training radiologists now" against Agarwal et al. (2024), where radiologists given AI recommendations were sometimes less accurate than alone because they over weighted the machine where they themselves were right (a Bayesian updating failure; users must know how the algorithm was trained); the STEM ads revisited; algorithmic pricing and antitrust (Amazon and Whole Foods, the Crane and Hovenkamp quotes, structural separation as the architectural remedy); algorithmic monoculture (Kleinberg and Raghavan, 2021: correlated failures, systematic exclusion, increased competition, collusion; CrowdStrike's patch grounding airlines; every firm's résumé screener chasing the same candidates; star topologies are fragile, the internet is distributed); human bias (judges ignoring accurate bail algorithms and going with a more biased gut, so audits must cover the human reaction too); governance by law (enforcement needs an audit system; GDPR penalties), architecture (structural separation, open source, blockchain), social norms (no LLMs in classrooms, watermarks on AI images) and markets (data exchanges with incentives not to abuse data).

**Mentored session and case study.** The quiz: the lifecycle starts with data collection ("good data leads to good AI"); selection bias is training data that fails to represent all groups; a hiring tool preferring men because of

historical data is historical bias; correlation shows a relation, causation shows direction and effect (umbrellas do not cause rain); SPD measures the difference in positive rates; counterfactual fairness means the prediction would not change if the sensitive attribute changed. The case was **Fair Gen**, an AI driven car insurance claim assessment in Google AI Studio with Gemini 2.5 Pro at temperature 0 and strict JSON output, on 100 claims with 14 columns (driver ages and names, vehicles, date, location, weather, accident type, two driver statements, police report summary). Phase A: an "expert AI insurance claim assessor" prompt returns at fault party, fault percentages summing to 100, and Approved or Rejected, with rejection rules for illegal activity (racing, DUI, unlicensed driving, hit and run) and five worked examples (a rear end collision with Driver B cited, a shared parking lot fault, a left turn on a flashing yellow arrow confirmed by dashcam, a deer swerve with no fault and a rejection, and a racing pair rejected). Phase B: an "AI fairness auditor" prompt receives the human and LLM outcomes, engineers its own features from the text, computes SPD per group, flags groups beyond $\pm 0.1$, and reports. Results: overall approval humans 60 percent, LLM 51 percent; disputed accident types humans 81 percent, LLM 26 percent; claims with conclusive evidence humans 61 percent, LLM 73 percent. Diagnosis: humans show leniency or confirmation bias toward their own insured (claims C0052, C0057, C0061 where a traffic camera showed the insured ran the stop sign, C0064 a DUI approved by oversight, C0074 a left turn lie caught by camera); the LLM shows a systematic, transparent evidence bias that penalizes claims without documentation. Recommendations: a confidence score with mandatory human review below a threshold, a tiered human in the loop process (automate clear cut claims, route 30 to 70 percent shared fault cases to adjusters with the LLM analysis attached), explainability by citing the decisive evidence, NLP to structure the facts first, and a feedback loop from escalated cases. Conclusion: a hybrid model, "the speed and consistency of AI with the judgment and compassion of human experts."

**Your notes.** Your only personal file in this module is the May 4 recording; it contains the professor's words with speech recognition slips ("Splinter and Alton", "Holland camp" for Hovenkamp, "Daniel green" for Daniel Crane, a stray "Hola") and no comments of your own. No quiz answers or scores are stored.

## Self paced module A. Data exploration: unstructured data

**What it taught.** Text as data: tokenization, stop words, bag of words and TF IDF, sentiment analysis, topic modeling, word embeddings, and a preview of how large language models changed text analytics. Released April 23, 2026 with an optional quiz.

**The mathematics under it.** TF IDF weights a term $t$ in document $d$ by $\mathrm{tf}(t,d) \times \log(N/\mathrm{df}(t))$. Latent Dirichlet allocation models each document as a mixture of topics and each topic as a distribution over words. Word embeddings place words in $\mathbb{R}^d$ so that cosine similarity reflects co occurrence, which is the same idea as the RAG embeddings of Module 11 applied to single words.

Your archive contains no folder for this module, so it cannot be checked against the delivered content. The brochure lists it as text processing, sentiment analysis and text classification, and its RapidMiner tooling (Text Processing extension: tokenization, stemming, stop word filtering, n grams) is the one you already used in the Yelp content based recommender.

## Self paced module B. Data exploration: temporal data

**What it taught.** Time series: trend, seasonality, stationarity, autocorrelation, moving averages, exponential smoothing, ARIMA style models, and forecasting evaluation with a time ordered split. Released April 23, 2026 with an optional quiz.

**The mathematics under it.** Autocorrelation at lag $h$ is $\rho(h) = \mathrm{Cov}(x_t, x_{t+h})/\mathrm{Var}(x_t)$. Simple exponential smoothing is $s_t = \alpha x_t + (1-\alpha)s_{t-1}$. An AR($p$) model is $x_t = c + \sum_{i=1}^{p} \phi_i x_{t-i} + \varepsilon_t$, and differencing turns a unit root series into a stationary one. For you, this module overlaps with what you know from markets; the main addition is the discipline of walk forward validation, never a random split, on temporal data.

No folder for this module exists in your archive either. The brochure lists it as time series methods and ARIMA modeling; Dahleh's generative AI lecture used the AR(p) process as an example of a generative model and mentioned simulated bank data for systemic risk analysis, which is the closest the core modules came to it.

---

# Part 4. Tools reference

## 4.1 RapidMiner AI Studio 2025.1 (Altair)

- **Why this version.** The case study videos were recorded on 2025.1 and the interface changed in later versions, so the office asked learners to install exactly 2025.1. (Some case videos were recorded on 9.10 and the walkthrough documents describe 9.10; Georg said 2026 works too. The educational licence is free for one year.)
- **Canvas vocabulary.** Repository (data and processes), Operators panel, Process canvas, Parameters panel, Results view. Operators connect through ports; the top left "inp" and right "res" ports are the process input and output. (The walkthrough adds the Views panel with Design, Results, Turbo Prep, Auto Model and Deployments, the Help panel with synopsis, description, parameters and tutorials, the Recommended Operators strip, breakpoints before and after an operator, and Auto Wire.)
- **A standard supervised flow.** Retrieve, Select Attributes, Replace Missing Values, Nominal to Numerical, Normalize, Set Role (label), Split Data (or Cross Validation), Learner (Decision Tree, Random Forest, Gradient Boosted Trees, Logistic Regression, Neural Net), Apply Model, Performance.
- **A standard unsupervised flow.** Retrieve, Normalize, k Means (or PCA), Cluster Model Visualizer.
- **Auto Model.** A wizard that runs the full flow for you and ranks models; the course used it for quick comparison but asked you to build the manual flow for the projects.

> - **Extensions you needed.** Operator Toolbox (outlier detection, extra performance criteria, Local Interpretation, SMOTE, and the Merge Attribute function), Text Processing (tokenization, stemming, stop words, n grams), and the Recommender Extension version 5.1.2 (Item k NN, User k NN, Matrix Factorization, top N recommendation). Install from Extensions, Marketplace, search, "Select for installation", accept, restart; the operators then appear in the Operators tab.
> - **Operators actually used in the course** (in addition to the flows above): Nominal to Numerical with dummy coding and comparison groups, Multiply, Correlation Matrix, Subprocess, Multiply Model, Performance (Regression) with RMSE and squared correlation, Optimize Parameters (grid search on criterion, depth, minimal leaf size, minimal size for split), Bagging, Loop for K fold and bootstrap validation, Aggregate, Join, Cross Distance, Generate Attributes, Nominal to Date, Filter Examples, Numerical to Polynominal, Write Excel and Write CSV, and the H2O based Neural Net.
> - **Common errors and fixes.** "Required parameter missing" on Retrieve means the repository location is unset; "illegal comparison group" means a typo in the category name; "Missing plugin" on a recommendation file means the Recommender extension is absent; the empty Training Resources folder is filled from the RapidMiner academy download; date parsing needs the format and locale set in Nominal to Date.

## 4.2 KNIME Analytics Platform

- **Vocabulary.** Node repository, workflow editor, node status lights (red, yellow, green), output ports, and views.
- **Equivalent nodes.** CSV Reader, Column Filter, Missing Value, One to Many (encoding), Normalizer, Partitioning, learners and predictors, Scorer, Numeric Scorer, ROC Curve, k Means, PCA, Keras Network Learner and Executor.
- **The hands on session** of March 22, 2026 built a neural network flow with the Keras nodes and compared it with a tree model on the same partitioning. (Correction: it used the RProp MLP Learner, not Keras; see Module 7.)

> - **Versions.** The course asked for 5.4.x (the videos show 4.7); you ran 5.9.0 on macOS. Workflows travel as `.knwf` files (File, Import KNIME Workflow) and the Optimization extension must be installed before a Parameter Optimization Loop will run.
> - **Nodes you actually used.** CSV Reader, Excel Reader, Data Explorer, Statistics, Value Counter, Bar Chart, Box Plot, Scatter Plot, Linear Correlation, Missing Value, Category to Number, One to Many, Rule Engine, String Manipulation, String Replacer, String To Number, Cell Splitter, Column Filter, Row Filter, Normalizer and Normalizer (Apply), Denormalizer, Partitioning and Table Partitioner (relative 70 percent, stratified, seeded), SMOTE, Decision Tree Learner and Predictor (gain ratio, MDL or no pruning), Random Forest Learner and Predictor, RProp MLP Learner and MultiLayerPerceptron Predictor (hidden layers, neurons per layer, max iterations, seed), Table Creator, Parameter Optimization Loop Start and End, Table Row to Variable, Scorer (confusion matrix, accuracy, Cohen's kappa, per class precision and recall), Numeric Scorer, Expression, Concatenate, GroupBy, Joiner, Sorter, Math Formula, Python Script, Spark Collaborative Filtering Learner.
> - **Habits the sessions insisted on.** Split before fitting any transformation and apply the fitted Normalizer to the test partition; use a static random seed so results are reproducible (results differ across learners because splits and initializations differ); annotate the canvas with colored sections (right click, New Annotation); Execute All with Shift F7; Java heap errors on big tables are fixed in `knime.ini`.

## 4.3 Teachable Machine, Ikigai, and generative tools

- **Teachable Machine** trains image, sound, or pose classifiers in the browser by transfer learning and exports the model. (A Google experiment launched in 2017; it reserves 15 percent of the data for validation and shows per class accuracy, the confusion matrix and the accuracy and loss curves under "Under the hood"; the defaults are 50 epochs, batch 16, learning rate 0.0001.)
- **Ikigai** (cofounded by Devavrat Shah) is a no code platform for tabular data, time series forecasting, and reconciliation used in the recommendation and temporal material. (Video 1.9 of the recommendation module is a hands on session on MovieLens in Ikigai, with a real time app that updates film similarities.)
- **ChatGPT, Gemini, NotebookLM, DALL E** were the generative tools; NotebookLM was the no code way to do retrieval augmented question answering on uploaded documents.

## 4.4 The generative and agentic stack you actually used

| Tool | Where | What for |
|---|---|---|
| Google AI Studio, Gemini 2.5 Pro | Modules 11 and 12 | System instructions plus few shot examples for classification and summarization; temperature 0 and strict JSON for the Fair Gen claim assessor and auditor |
| NotebookLM | Module 11 practice | Grounded question answering on an HR manual with clickable citations |
| n8n | Modules 10 and 11, MediSure project | Visual workflow automation: Chat Trigger, AI Agent, chat model and memory sub nodes, tools, sub workflows, Structured Output Parser, HTTP Request, Code, Edit Fields, Aggregate, Wait, Schedule Trigger, Markdown, email nodes; the Great Learning lab, and your own instance on Hostinger |
| OpenAI models | n8n | `gpt-5-mini` as the MediSure agent, `gpt-4.1` as its judge, GPT 4.1 Mini as the session evaluator, `text-embedding-3-large` (3,072 dimensions) for MediSure and `text-embedding-3-small` (1,536) for NovaMind |
| Pinecone | MediSure | Serverless dense index, cosine metric, free Starter tier (2 GB, 1 million read and 2 million write units per month), 224 vectors |
| Weaviate Cloud | NovaMind session | Sandbox cluster and collection ArxivArticles as insert target and retrieval tool |
| DuckDuckGo community node | Web search sub workflow | Free search without an API key; needs a wait between calls and returns variable results |
| Adobe Firefly (Veo 3.1), ElevenLabs, Qooly | Browbake exercise | Text to video, text to speech, merge |
| Mistral Pixtral | Yoga case study | Zero shot vision language classification as a comparison model |
| Otter, Granola, Apple Notes transcription | Your own workflow | Recording and transcribing live sessions and lecture videos |

# Part 5. Your projects and portfolio

| Project | Module | Tool | Deliverable | Status |
|---|---|---|---|---|
| Predicting the cancellation of Hotel Bookings by customers | 5, classification | RapidMiner or KNIME flow, written report | Classification models, metric comparison, recommendations | In ePortfolio since March 17, 2026 (submitted March 8, 2026 as a KNIME hands on deck for INN Hotels Group; pruned random forest chosen on recall, about 84 percent of cancellations detected) |
| Project 6, Prediction Methods with Deep Learning and Neural Networks | 9 | RapidMiner or KNIME flow | Neural network versus classical models on a business dataset | Due April 20, 2026 (built March 20 to 28, 2026 in KNIME on the HMEQ loan data; SMOTE plus a 770 run grid; accuracy 90.3 percent, recall 63.0 percent, kappa 0.664) |
| Business Applications of Generative AI course project | 11 | Generative tools, workflow description | Use case, prompts or RAG design, evaluation, risks | Due May 4, 2026 (MediSure Regulatory Intelligence Workflow, four n8n workflows on your Hostinger server with Pinecone, `gpt-5-mini`, DuckDuckGo and a `gpt-4.1` judge; final PDF exported May 2, 2026; four evaluation tests scored 5 of 5 on every dimension) |
| Project Discussion Forum | across modules | Olympus forum | Scored contributions | Scores published March 28, 2026 |

Your ePortfolio on Olympus holds these projects and can be shared publicly; the sample ePortfolio the advisor sent shows the format.

> **Case studies you worked through** (the brochure promises 20 or more; these are the ones in your archive): Uber New York pickups; Melbourne housing; the HIP mammography trial; US colleges (PCA); country clustering; marketing customer segmentation; ShowTime viewership; Hospital length of stay; BigMart sales; Sales Leads

> conversion; Online Shopper intention; Credit card churn; INN Hotels cancellations; Reader's Club books; GoodBooks; Yelp; MovieLens; Fashion MNIST (trees, forest, ANN and CNN); Insurance Charges; University Admission; HMEQ loans; rice grain classification; your own Yoga Pose Detection; MarketPulse web search agent; Browbake marketing assets; NovaMind arXiv trends; hotel review classification; patient friendly summaries; the HR policy assistant; MediSure; Fair Gen insurance claims; and your own Netflix Challenge workflow.
>
> **What is missing from the archive.** No grade, mark, evaluator comment or quiz score for any assessment or project; no files for the two self paced modules; no recording of the weekly quizzes except your Data Exploration answer key. If you want the marks in this book, export the Olympus grade page and the ePortfolio entries.

**What came next.** After this program you completed the AI Native Professional program (Workflows and Agents for Productivity, June to August 2026) and part of the UT Austin postgraduate program in AI Agents for Business Applications. Those are outside the scope of this book, but the projects from them (MediSure regulatory workflow, GlobalEdge RAG proof of concept, resume shortlisting agent, last mile delivery exception handling) are the direct continuation of Modules 10 and 11 here. (Correction: the MediSure workflow was the graded Project 3 of this program, built in late April 2026, not a later project.)

---

# Part 6. Formula sheet and glossary

## 6.1 Formula sheet

| Topic | Formula |
|---|---|
| Empirical risk | $\hat{R}(\theta) = \frac{1}{n}\sum_i \ell(f_\theta(x_i), y_i)$ |
| OLS | $\hat{\beta} = (X^\top X)^{-1} X^\top y$; ridge $(X^\top X + \lambda I)^{-1} X^\top y$ |
| Logistic | $p = \sigma(x^\top \beta)$, $\sigma(z) = 1/(1+e^{-z})$, coefficient = log odds ratio |
| Gini, entropy | $1 - \sum_c p_c^2$; $-\sum_c p_c \log_2 p_c$ |
| Precision, recall, F1 | $\frac{TP}{TP+FP}$, $\frac{TP}{TP+FN}$, $\frac{2PR}{P+R}$ |
| Cost threshold | predict 1 if $\Pr(y=1 \mid x) > \frac{c_{FP}}{c_{FP}+c_{FN}}$ |
| K means | $\min \sum_j \sum_{x \in S_j} \lVert x - \mu_j \rVert^2$ |
| GMM | $p(x) = \sum_j \pi_j \mathcal{N}(x; \mu_j, \Sigma_j)$, fit by EM |
| PCA | eigenvectors of $\frac{1}{n} X^\top X$; variance explained $\sum_{i \le m} \lambda_i / \sum_i \lambda_i$ |
| t SNE | minimize $\sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$ |
| Matrix factorization | $\min_{U,V} \sum_{\text{obs}} (r_{ui} - u_u^\top v_i)^2 + \lambda(\lVert U \rVert_F^2 + \lVert V \rVert_F^2)$ |
| Cosine similarity | $q^\top e / (\lVert q \rVert \lVert e \rVert)$ |
| Neural layer | $h^{(l)} = \phi(W^{(l)} h^{(l-1)} + b^{(l)})$; backprop $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(z^{(l)})$ |
| Softmax, cross entropy | $\hat{y}_c = e^{z_c} / \sum_k e^{z_k}$; $\ell = -\sum_c y_c \log \hat{y}_c$ |
| Convolution | $(K * I)(i,j) = \sum_{a,b} K(a,b) I(i+a, j+b)$ |
| Attention | $\mathrm{softmax}(QK^\top / \sqrt{d_k}) V$ |
| Temperature | $p_i \propto \exp(z_i / \tau)$ |
| TF IDF | $\mathrm{tf}(t,d) \log(N / \mathrm{df}(t))$ |

| Topic | Formula |
|---|---|
| Autocorrelation | $\rho(h) = \mathrm{Cov}(x_t, x_{t+h}) / \mathrm{Var}(x_t)$ |
| AR(p) | $x_t = c + \sum_{i=1}^{p} \phi_i x_{t-i} + \varepsilon_t$ |

| Topic (added) | Formula |
|---|---|
| Binomial test (HIP study) | deaths $\sim \mathrm{Bin}(31000, 0.002)$; p value $= \Pr(X \le 39) = 0.0012$; reject below 0.05 |
| Confidence interval, Wald test | $\hat{\theta}_j \pm 2\,\mathrm{SE}_j$; reject $\theta_j = 0$ if the interval excludes 0 |
| $R^2$ from sums of squares | $R^2 = 1 - \mathrm{RSS}/\mathrm{TSS}$, $\mathrm{TSS} = \sum (y_i - \bar{y})^2$, $\mathrm{RSS} = \sum (y_i - \hat{y}_i)^2$ |
| Dummy coding rule | one dropped category per nominal variable, because $\sum_k d_k = 1$ is collinear with the intercept |
| Information gain | $\mathrm{IG}(Y, X) = H(Y) - H(Y \mid X)$ |
| Majority vote error | with $B$ independent errors of rate $q < 1/2$, the vote errs only if more than $B/2$ err |
| Cohen's kappa | $\kappa = (p_o - p_e)/(1 - p_e)$ |
| Silhouette | $s_i = (b_i - a_i)/\max(a_i, b_i)$ |
| BIC | $k \ln n - 2 \ln \hat{L}$ |
| Mean centered cosine | subtract each user's mean before the cosine; users with no common items get similarity 0 |
| Averaging estimator error | $\pm 1/\sqrt{n}$ (central limit theorem) |
| Singular value thresholding | $\hat{L}_{ij} = \frac{1}{p} \sum_{k=1}^{r} s_k u_{ik} v_{jk}$; rank $r$ needs about $(m+n)r$ entries |
| Alternating least squares | fix $V$, solve $\min_U$ by regression; fix $U$, solve $\min_V$; repeat |
| ReLU gradient, one hidden layer | $\partial L / \partial w_{11} = x_1 \cdot 1[z_1 > 0] \cdot v_1 \cdot (F(x_i) - y_i)$ |

| Topic (added) | Formula |
|---|---|
| Weight decay | $w_{t+1} = w_t(1 - \eta\alpha) - \eta\,\partial L/\partial w$ |
| He initialization | $w \sim \mathcal{N}(0,\ 2/\text{fan in})$ |
| Dropout at test time | scale weights by $1 - r_d$ (equivalently scale activations by $1/(1 - r_d)$ in training) |
| Convolution output size | $\lfloor (n + 2p - f)/s \rfloor + 1$; same padding $p = (f - 1)/2$ |
| CNN parameters | $F$ filters of $f \times f \times C$: $F(f^2C + 1)$ parameters, independent of image size |
| Attention (Dahleh's form) | $A_{ij} = \langle x_i, K_1 x_j + K_2 s_{ij} \rangle$; $x_i \leftarrow x_i + \sum_j \frac{e^{A_{ij}}}{\sum_k e^{A_{ik}}} V x_j$ |
| LLM as sparse completion | $50000^{100}$ possible 100 word prompts over a 50,000 word vocabulary |
| Statistical parity difference | $\Pr(\hat{y} = 1 \mid \text{group}) - \Pr(\hat{y} = 1 \mid \text{overall})$; flag beyond $\pm 0.1$ |
| Counterfactual fairness | decision unchanged when only the protected attribute is flipped |
| SMOTE | synthetic minority rows interpolated between a minority point and one of its $K$ nearest minority neighbors; rate 2 doubles the class |
| RAG chunking used | 1,000 characters with 200 overlap (about 250 tokens, 20 percent overlap); top 5 chunks; index dimension must equal the embedding dimension |

## 6.2 Glossary

- **Agent.** A model in a loop that plans, calls tools, observes results, and continues until a goal is met.
- **AUC.** Area under the ROC curve; probability that a random positive outranks a random negative.
- **Backpropagation.** Chain rule computation of loss gradients layer by layer.
- **Bagging, boosting.** Averaging models trained on bootstrap samples (variance reduction) versus fitting models sequentially to residual errors (bias reduction).
- **CEU.** Continuing education unit; ten contact hours per unit.
- **Cold start.** A recommendation problem for users or items with no interaction history.
- **Context window.** The maximum number of tokens a language model can attend to at once.
- **Cross validation.** Rotating the validation fold across the data to estimate generalization error with less variance.

- **Embedding.** A dense vector representation of a word, chunk, image, or user learned so that similarity is meaningful.
- **Feature importance.** A ranking of inputs by their contribution to a model's predictions; SHAP values give a per prediction version.
- **Hallucination.** A fluent but false statement by a generative model.
- **Hyperparameter.** A setting chosen before training (depth, learning rate, $k$, $\lambda$), tuned on validation data.
- **Mentored Learning Session (MLS).** The weekly mandatory live session with the Great Learning mentor, scored for attendance.
- **Olympus.** Great Learning's learning management system.
- **Operator, node.** The unit of work in RapidMiner and KNIME respectively.
- **Overfitting.** Low training error with high test error; the model has memorized noise.
- **RAG.** Retrieval augmented generation; retrieve relevant text, then generate with it in the prompt.
- **Regularization.** A penalty on model complexity added to the loss.
- **Stationarity.** Constant mean and autocovariance over time; required by ARIMA style models.
- **Transfer learning.** Reusing a pretrained network and retraining only its last layers.

- **Algorithmic monoculture.** Many organizations running the same algorithm, which produces correlated failures, systematic exclusion and needless competition (Kleinberg and Raghavan, 2021; CrowdStrike; shared résumé screeners).
- **Auto Model, Turbo Prep.** RapidMiner views that automate model comparison and interactive data preparation.
- **Business Interpretation track.** The project route in which you interpret the provided screenshots instead of running the tool.
- **Comparison group.** The category dropped in dummy coding; the other coefficients are read relative to it.
- **Confounder.** A variable that causes both treatment and outcome (the sun for ice cream and suntan); randomization removes it.
- **Counterfactual fairness.** An individual level fairness test: would the decision change if only the protected attribute changed.
- **Data operator.** Shah's name for the person who reconciles several systems of record into a source of truth and decides daily.
- **Differential privacy.** A computation whose output barely changes when any one record is removed.
- **Grounding.** Whether an answer is supported by the retrieved sources; one of the four judge dimensions with relevance, completeness and clarity.
- **Induction head.** A two layer attention pattern that copies the token that followed an earlier occurrence of the current token; the mechanism behind in context learning.
- **Kappa (Cohen's).** Agreement between predictions and truth beyond chance, reported by KNIME's Scorer; 0.6 to 0.8 is "substantial".

- **LLM as a judge.** A second model that scores another model's answer on a rubric; better from a different model family.
- **Leave one out (LOOCV).** K fold cross validation with $K = n$.
- **Parameter Optimization Loop.** KNIME's grid search construct (Table Creator, Loop Start, learner, predictor, scorer, Table Row to Variable, Loop End) with an objective such as kappa.
- **Pose project.** Teachable Machine mode that classifies body keypoints from a pretrained pose estimator rather than raw pixels.
- **RProp MLP.** KNIME's resilient backpropagation multilayer perceptron; needs a nominal class column and numeric inputs.
- **Second price auction.** The ad market mechanism in which the highest bidder wins and pays the second highest bid; agnostic in itself, biased through the perceptions that set the bids.
- **SMOTE.** Synthetic Minority Oversampling Technique; applied to the training partition only.
- **SPD.** Statistical parity difference; see the formula sheet.
- **Structural separation.** The architectural remedy for platform antitrust: the owner of the platform should not compete on it.
- **Sub workflow (n8n).** A workflow with a typed input contract that another workflow, or an agent, can call as a tool once published.
- **Vector store.** A database indexed by embedding vectors (Pinecone, Weaviate); the index dimension is fixed at creation.
- **Watermark code.** The learner id 8H4K6FA53C stamped with your email on every Great Learning PDF.

---

# Part 7. Self test and revision plan

## 7.1 Twenty questions

1. Why must features be standardized before K means and PCA but not before a decision tree?
2. Write the K means objective and explain why Lloyd's algorithm can stop at a local minimum.
3. What does a Gaussian mixture give you that K means does not?
4. State the PCA solution in terms of the covariance matrix and in terms of the SVD.
5. Why are distances between clusters in a t SNE plot not interpretable?
6. What does a zero in the precision matrix of a Gaussian mean?
7. Derive the OLS estimator from the normal equations. What happens when $X^\top X$ is singular, and how does ridge fix it?
8. Interpret a logistic regression coefficient of 0.69.
9. Compute Gini and entropy for a node with class proportions 0.8 and 0.2.
10. Why does a random forest reduce variance, and why does gradient boosting reduce bias?
11. Given costs $c_{FN} = 5$ and $c_{FP} = 1$, what probability threshold should a classifier use?
12. Why is accuracy misleading on the hotel cancellation data if only 37 percent of bookings cancel? (Your data: 32.8 percent.)
13. Write the matrix factorization loss for recommendations and explain why only observed entries appear.
14. What is the cold start problem and give one content based remedy.
15. Write the backpropagation recursion for one hidden layer.
16. Why does ReLU help deep networks train?
17. Explain parameter sharing in a convolution and its effect on the number of parameters.
18. What does transfer learning freeze, and why does Teachable Machine need so few examples?
19. Write the attention formula and explain the role of $\sqrt{d_k}$.
20. In a RAG system, name three settings that control retrieval quality and one metric for faithfulness.

## 7.2 Twenty more questions, taken from what the course actually examined

21. In the HIP study, why is comparing the screened women with the women who refused invalid, and what number proves it?
22. A p value of 0.0012 was obtained. State exactly what it is the probability of, and what it is not.

23. The advertising regression gives a negative newspaper coefficient while the simple regression gives a positive one. Explain both.
24. Why must one category be dropped when dummy coding a nominal variable for linear regression, and which category did the ShowTime session drop for each variable?
25. Distinguish the coefficient of a regressor from its p value in one sentence each, and give the ShowTime example of a real but small effect.
26. Define K fold cross validation, leave one out and the bootstrap, and say which one gives a standard error for a coefficient.
27. Compute the information gain of a split that sends a 50/50 node into two pure children.
28. Why did the Sales Leads deck prefer the pruned tree to the pruned forest, and what rule of thumb for overfitting did it use?
29. In your Project 1, why was recall for the cancelled class the key metric, and what did the default tree's 243 false negatives mean for the hotel?
30. Write the cosine similarity of the users with ratings $(3, 4, *, 2)$ and $(*, 4, 5, 1)$ before and after mean centering.
31. Why does singular value thresholding rescale by $1/p$, and what does a smooth singular value decay tell you?
32. On the Reader's Club data, item item CF beat SVD (RMSE 1.514 versus 1.896). Give two reasons the deck offered.
33. State the vanishing gradient problem in terms of the product that every gradient is, and name the initialization that mitigates it for ReLU.
34. In your HMEQ project, what did SMOTE change in the training set, why was it applied only to the training port, and what did it cost in precision?
35. Explain why your Project 6 base model numbers in the deck (kappa 0.327) differ from the KNIME screenshot (kappa 0.385), and which you would report.
36. For a $6 \times 6 \times 3$ image, compare the parameter count of a dense layer of 10 units with that of 10 filters of $3 \times 3 \times 3$.
37. In your yoga case, warrior scored 0.68 on validation and 100 percent on the held out test. Reconcile the two numbers.
38. Write Dahleh's attention score $A_{ij}$ and explain why embedding tokens in high dimension makes syntax linear.
39. In MediSure, why must the retrieval embedding model and dimension equal the indexing ones, and what happens otherwise?
40. Define SPD, compute it for the Fair Gen disputed claims (humans 81 percent, LLM 26 percent, overall about 60 and 51 percent), and say which bias each side shows.

## 7.3 A thirty day plan to go one level deeper

- **Days 1 to 5.** Reproduce the hotel cancellation project in Python (pandas, scikit learn). Match your RapidMiner metrics, then add a cost sensitive threshold and a calibration plot. (Your metrics were produced in KNIME: default tree 83.0 percent test accuracy and 73.4 percent recall, default forest 88.0 and 83.8; reproduce those first with a 70/30 split, then add the pruned versions whose numbers you only have as screenshots.)
- **Days 6 to 10.** Implement K means, PCA, and a Gaussian mixture from scratch with NumPy on the same data. Compare with scikit learn. (Use the country dataset: reproduce the K means clusters of 112, 51 and 5 and the K medoids clusters of 34, 39, 68 and 26.)
- **Days 11 to 15.** Implement matrix factorization by alternating least squares on the MovieLens 100k dataset and evaluate precision at 10 per user. (Then add the $1/p$ singular value thresholding estimator and check that ALS beats it, as the Ikigai session showed.)
- **Days 16 to 20.** Build the neural network of Project 6 in PyTorch, log training and validation loss, and beat it with a gradient boosted tree to see the tabular benchmark for yourself. (Target: recall above 63 percent on the Bad class at 90 percent accuracy on the HMEQ data with a stratified 70/30 split and seed 42; try the 0.35 threshold you recommended.)
- **Days 21 to 25.** Fine tune a small pretrained vision model on a two class image set of your own; compare with Teachable Machine. (Use your 1,081 yoga images and see whether a fine tuned CNN fixes the warrior class.)
- **Days 26 to 30.** Build a RAG assistant over your own documents (you already have the commodities RAG assistant and the GlobalEdge proof of concept), then measure retrieval recall and answer faithfulness on twenty hand written questions. (Reuse the MediSure judge prompt, and add the missing piece of that project: cross validation of chunk size and top k against the twenty questions.)

---

# Appendix A. Sources used for this book

**Emails in your inbox (Great Learning and MIT PE program office), in date order.**

- Dec 29, 2025: brochure and curriculum email; advisor next steps (fee USD 2,850, 11 modules, 80 hours, 6 to 12 hours per week, start January 17, 2026); offer for registration with attached offer letter and brochure; payment receipt; welcome from program manager Juhi.
- Dec 30, 2025: start your journey (Olympus, Program Overview, Pre work, orientation January 17); pre work session notice for January 4.
- Jan 2, 2026: program manager Ansuman; RapidMiner AI Studio 2025.1 requirement.
- Jan 6, 2026: online class notice, Mathematics and Statistics behind AI, January 11.
- Jan 17, 2026: recording of the orientation online class.
- Mar 3, 2026: mentored session notice, Recommendation Systems, March 7.
- Mar 15, 2026: mid program feedback reminder.
- Mar 16, 2026: hands on KNIME session notice, Neural Networks, March 22.
- Mar 17, 2026: ePortfolio, hotel booking cancellation project.
- Mar 28, 2026: project discussion forum scores.
- Apr 9, 2026: revision week, Project 6 deadline, support sessions, Week 9 Business Applications of Generative AI release.
- Apr 15, 2026: mentored session notice, Generative AI Foundations, April 18; Apr 16 course assessment reminder.
- Apr 22 and 23, 2026: mentored session notice for April 25; ninth session notice with assessment and project deadlines; self paced modules Unstructured Data and Temporal Data.
- May 1, 2026: reminders for the Ethical and Responsible AI assessment and the Business Applications assignment, both due May 4.
- May 26, 2026: certificate from Accredible.
- Jun 3, 2026: CEU letter.
- Jul 21, 2026: alumni survey.

**Public program descriptions.** MIT Professional Education course catalog page for the program; Great Learning program page and brochure summaries; third party listings that reproduce the week by week curriculum (Course Report, Educatly, Northwest Education, Shortcoursesportal, Scribd copies of the brochure).

> **Your Google Drive archive (second edition).** The folder *MIT* shared on September 10, 2026, read in full through the Drive connector: 11 course folders and the admission folder as described in section 1.5, about 300 files. Official documents: MIT NCAI Intl brochure (September 2025 version), admissions offer letter of December 29,

2025, "2026 Delivery Schedule NCAI INTL", orientation session deck, CEU letter, Accredible certificate and its W3C and Open Badges JSON exports, "Meet Your Faculty" page. Course materials: module outlines, learning instrument pages, faculty pages, MIT lecture handouts and transcripts (Data Exploration 1.2 to 1.9, Regression 1.0 to 1.16, Decision Systems 1.1 to 1.8, Recommendation 1.1 to 1.9, Neural Networks 1.1 to 1.15, Computer Vision reader, Generative AI and Business Applications page exports, Ethical and Responsible AI reader, Dahleh's three lecture decks), Great Learning mentor session decks and session plans for every module, hands on session plans, practice case decks, problem statements, walkthroughs, FAQs (regression, evaluation metrics, classification, random forest, recommendation, neural networks, CNN, hypothesis testing), project briefs, templates and track descriptions, the NovaMind and MarketPulse n8n decks, the Browbake and Business Applications practice exercises, the Fair Gen no code solution, the RapidMiner and KNIME interface walkthroughs and extension guides. Your own files: transcripts (orientation, January 18, January 25, February 14, February 15, May 4), summaries and notes for every module, the Data Exploration quiz answer key, the Project 1 deck with drafts and workflow notes, the Project 6 deck, reports, dashboards, assembled screenshots and AI chat transcript, the Netflix Challenge guide in French and English, the Browbake outputs and web tool, the yoga case study decks and charts, the CNN study guide and Granola summary, the MediSure final PDF, seven deck versions, plan, test notes, Pinecone recipe, raw URL list and corpus PDFs, and the three credential triptychs.

**What could not be verified.** The Olympus course pages themselves remain unreachable from the writing environment; every Olympus fact above comes from your exports. Unreadable files: *ai book 2.pdf* (185 MB, no text layer), the computer vision *Lecture Slides.pdf* (143 MB), the optional recommendation lecture slides (image only), the two yoga LinkedIn cards and the English comparison chart (images), the NovaMind solution workflow (mhtml), all videos, audio files and zip archives. Truncated in extraction: your Project 6 chat transcript (stops after about 50,000 characters), your compiled recommendation notes (stop in the Yelp results), the February 14 closed captions (stop at the start of model building), the January 18 transcript (stops in quiz question 3), and a few handouts that lose their last sentence. No file contains grades or quiz scores.

# Appendix B. Reading list matched to the modules

- Modules 2 to 7: *An Introduction to Statistical Learning* (James, Witten, Hastie, Tibshirani), chapters on regression, classification, tree methods, and unsupervised learning. Free PDF from the authors.
- Module 6: Koren, Bell, Volinsky, "Matrix Factorization Techniques for Recommender Systems", IEEE Computer, 2009.
- Modules 7 and 8: *Deep Learning* (Goodfellow, Bengio, Courville), chapters 6 and 9. Free online.
- Module 10: Vaswani et al., "Attention Is All You Need", 2017, and the Illustrated Transformer blog post for the pictures.
- Module 11: Lewis et al., "Retrieval Augmented Generation for Knowledge Intensive NLP Tasks", 2020.
- Module 12: NIST AI Risk Management Framework 1.0, and Kleinberg, Mullainathan, Raghavan, "Inherent Trade Offs in the Fair Determination of Risk Scores", 2016.
- Temporal data: Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*, free online.

**Readings the course itself pointed to** (curated resource pages and lecture references in your archive):

- Module 1: "The Rise of Artificial Intelligence in Healthcare Applications" (Elsevier chapter); MIT Technology Review with JPMorgan Chase, "How AI is powering the future of financial services" (video); MIT Technology Review with Microsoft, "Unleashing the power of AI for education"; G2, "Democratizing AI with Low Code and No Code Machine Learning Platforms"; Fortune, "You can now put AI tools in the hands of all your employees. But should you?"
- Module 2: MIT Technology Review with Adobe, "Turning AI into your customer experience ally"; Frontiers in Computational Neuroscience, "The Application of Unsupervised Clustering Methods to Alzheimer's Disease"; Malaysian Online Journal of Educational Technology, "Discovery of Course Success Using Unsupervised Machine Learning Algorithms".
- Module 3: Scott Fortmann Roe, "Understanding the Bias Variance Tradeoff" (essay); the scikit learn algorithm cheat sheet shown in the session.
- Module 4: the CMU 10 601 decision tree notes cited in the mentor deck.
- Module 6: Töscher, Jahrer and Bell, "The BigChaos Solution to the Netflix Grand Prize" (2009), which you read in full; the GroupLens MovieLens datasets; Celma and Lamere (ISMIR 2007) on the share of consumption driven by recommendations (Netflix two thirds, Amazon 35 percent, Google News 38 percent more clicks).
- Module 7: Yoshua Bengio, "Practical Recommendations for Gradient Based Training of Deep Architectures"; Srivastava et al., "Dropout" (JMLR 2014); Rumelhart, Hinton and Williams (1986).

- Module 8: IBM Technology, "What are Convolutional Neural Networks"; 3Blue1Brown, "But what is a convolution?"; MIT 6.S191 lecture on CNNs by Alexander Amini (from your study guide).
- Module 12: Munther Dahleh, *Data, Systems, and Society: Harnessing AI for Societal Good* (Cambridge University Press); Agarwal et al. (2024) on radiologists with AI assistance; Kleinberg and Raghavan (2021) on algorithmic monoculture; Auten and Splinter (Journal of Political Economy) against Piketty on US inequality.

# Appendix C. Discrepancies between your own files

These are the points where two of your documents disagree, so that you know which number to quote.

1. **Project 6 base model.** Deck and reports: accuracy 82.9 percent, kappa 0.327, recall 29.7 percent, TP 106, FN 251, FP 54, TN 1,377 (dashboard) or 1,380 (deck). KNIME screenshot: accuracy 84.1 percent, kappa 0.385, recall 34.7 percent, TP 124, FN 233, FP 51, TN 1,380. The deck figures match the Business Interpretation template's reference kappa of 0.327, so they are the teacher's numbers. The improved model (225, 132, 42, 1,389, kappa 0.664) is consistent everywhere and is the number to quote.
2. **Project 1 pruned models.** Their metrics exist only as screenshots. The "about 84 percent of cancellations detected" claim equals the unpruned forest's test recall of 83.8 percent.
3. **Project 1 date.** Title slide March 3, file export March 8, deadline March 8 (schedule) or March 9 (as you counted it).
4. **Project deadlines.** Project 2: April 19 (schedule), April 20 12:30 EST (office email), April 20 2:30 PM (problem statement). Project 3: May 10 (schedule), May 4 (office email and problem statement). Business Applications quiz: April 26 (schedule), April 27 (office email).
5. **Week numbering.** The orientation deck and brochure count 12 weeks with two project weeks; the office notices count graded courses ("Week 9 content" for Business Applications, "Weeks 11 and 12" for the self paced modules); the mentored session of April 25 was the ninth session.
6. **Program end date.** May 3, 2026 on the CEU letter (last quiz deadline), May 20 on the credential (completion date), May 26 on the certificate email.
7. **Faculty titles.** Jegelka is "Associate Professor" in the brochure and "X Consortium Career Development Associate Professor" in the orientation deck; Uhler is "joint EECS and IDSS" in the brochure and "Henry L. and Grace Doherty Associate Professor" elsewhere; Shah is also Director of the Statistics and Data Science Center in the faculty page.
8. **Pre work quiz.** The pre work page says a mandatory quiz was due the weekend after orientation and, lower down, that the course carries no assessment counting toward the score. Both statements are in the source.
9. **Yoga comparison chart.** The French bar chart pairs Teachable Machine's validation accuracies with Pixtral's held out test accuracies, which your own deck flags as not a matched benchmark.

10. **The MediSure plan versus the build.** Planned `gpt-4o`, a paid search API and the disk reader; built `gpt-5-mini`, DuckDuckGo and GitHub raw URLs. The plan foresaw a replica in the Great Learning lab; the report documents the Hostinger build only.
11. **HMEQ missing value strategy.** The chat of March 21 planned median for DELINQ, NINQ and CLNO and a fixed 0 for DEROG; the analysis report states median for all numeric columns and mode for the categorical ones.
12. **Data Exploration quiz answer 15.** Your answer key records a corrected answer ("all of the above") with the note that the original was too narrow; whether the correction was made before or after submission is not recorded.